

Thiết Kế Cơ Sở Dữ Liệu

Đề Tài: LivingDocs

Nhóm 1

2026

Mục lục

Giới Thiệu Đề Tài	v
I Thiết Kế Quan Niệm	1
1 Thuật toán 2.1 & Mô hình Conceptual ERD	3
1.1 Phân tích Nghiệp vụ: Phạm vi, Thực thể & Ràng buộc	4
1.1.1 Khoanh vùng phạm vi lưu trữ (Scope Boundaries) . .	4
1.2 Từ điển nghiệp vụ (Data Dictionary) – chuẩn hóa tên gọi . .	6
1.3 Phân loại Tác nhân & Phạm vi Nghiệp vụ (Actor Classification & Business Scope)	8
1.3.1 Danh mục Tác nhân Hệ thống (System Actors)	8
1.3.2 Chi tiết các Thực thể Dữ liệu Cốt lõi (Core Data Entities)	11
1.4 Chi tiết các Thực thể Dữ liệu	13
1.4.1 Nhóm Quản trị & Kết nối	13
1.4.2 Workspace (Thực thể mạnh)	14
1.4.3 Role (Thực thể mạnh)	15
1.4.4 Nhóm Mã nguồn & Cấu trúc Code	17
1.4.5 Nhóm Khuôn mẫu Tài liệu (Template Engine Domain)	21
1.4.6 Nhóm Tài liệu & Vòng đời Dữ liệu (Documentation Domain)	23
1.4.7 Nhóm Kiểm duyệt & Phê duyệt (Review & Governance Domain)	24
1.4.8 Nhóm AI Evidence, Giám sát & Kiểm toán	26
1.4.9 Thực thể mối kết hợp / Sự kiện: Approval	26
1.5 Mô hình hóa Mối kết hợp & Bản số Dữ liệu	27
1.5.1 Nhóm Quản trị & Kết nối (Auth & Workspace Domain)	27
1.5.2 Mối kết hợp 3 ngôi: User — Workspace — Role	27

1.5.3	Mối quan hệ: Role — contains — Permission	28
1.5.4	Mối quan hệ: User — connects — Github_Connection	28
1.5.5	Nhóm Mã nguồn & Cấu trúc Code (VCS & Code Domain)	29
1.5.6	Mối quan hệ: Workspace — owns — Repository	29
1.5.7	Mối quan hệ: Repository — has_branch — Branch	29
1.5.8	Mối quan hệ: Branch — contains_commit — Commit	30
1.5.9	Mối quan hệ: User — makes_commit — Commit	30
1.5.10	Mối quan hệ: Repository — receives_pr — PullRequest	30
1.5.11	Mối quan hệ: Commit — generates_diff — CodeDiff	31
1.5.12	Mối quan hệ: PullRequest — includes_commit — Commit	31
1.5.13	Mối quan hệ: Commit — modifies — File	32
1.5.14	Mối quan hệ: Repository — contains_file — File	32
1.5.15	Mối quan hệ: File — parses_into — CodeEntity	32
1.5.16	Mối quan hệ: CodeEntity — depends_on — CodeEntity	33
1.5.17	Nhóm Khuôn mẫu Tài liệu (Template Engine Domain)	33
1.5.18	Mối quan hệ: Workspace — defines — Template	33
1.5.19	Mối quan hệ: Template — has_section — TemplateSection	34
1.5.20	Mối quan hệ: TemplateSection — contains_placeholder — Placeholder	34
1.5.21	Mối quan hệ: Placeholder — maps_to — CodeEntity	34
1.5.22	Nhóm Tài liệu & Vòng đời Dữ liệu (Documentation Domain)	35
1.5.23	Mối quan hệ: Workspace — manages — Document	35
1.5.24	Mối quan hệ: User — authors — Document	35
1.5.25	Mối quan hệ: Document — has_version — DocumentVersion	36
1.5.26	Mối quan hệ: DocumentVersion — tracks_change — ChangeLog	36
1.5.27	Nhóm Kiểm duyệt & Phê duyệt (Review & Governance Domain)	36
1.5.28	Mối quan hệ: DocumentVersion — has_review — Review	36
1.5.29	Mối quan hệ: User — performs_review — Review	37
1.5.30	Mối quan hệ: Review — contains_comment — ReviewComment	37
1.5.31	Mối quan hệ: DocumentVersion — requires_approval — Approval	38

1.5.32	Mối quan hệ: User — grants_approval — Approval .	38
1.5.33	Nhóm AI Evidence, Giám sát & Kiểm toán (AI Evidence & Observability Domain)	39
1.5.34	Mối quan hệ: DocumentVersion — has_evidence — GroundingEvidence	39
1.5.35	Mối quan hệ: GroundingEvidence — references_code — CodeEntity	39
1.5.36	Mối quan hệ: GroundingEvidence — detects — DriftAlert	40
2	Sơ đồ Quan niệm ERD / UML Class Diagram	41
2.1	Sơ đồ ERD mức khái niệm	42
2.2	Sơ đồ Class	43
II	Thiết kế Logic	45
3	Áp dụng Thuật toán 2.2 (Lược đồ Quan hệ & Chuẩn hóa) 3NF	47
3.1	Ánh xạ Thực thể, Mối kết hợp & Phẳng hóa Thuộc tính (Thuật toán 2.2)	48
3.2	Lược đồ Quan hệ Lô-gíc (Logical Schema) & 3NF-Phân hệ Quản trị & Xác thực	52
3.2.1	Lược đồ Cú pháp Rút gọn & Chứng minh Chuẩn hóa 3NF	55
3.2.2	Báo cáo Chứng minh Chuẩn hóa 3NF	56
3.3	Lược đồ Quan hệ Lô-gíc (Logical Schema) & 3NF-Phân hệ Mã nguồn & Cấu trúc Code	58
3.3.1	Lược đồ Dạng Cú pháp Rút gọn	61
3.3.2	Báo cáo Chứng minh Chuẩn hóa 3NF	62
3.4	Lược đồ Quan hệ Lô-gíc (Logical Schema) & 3NF-Phân hệ Mẫu & Tài liệu Cốt lõi	64
3.4.1	Lược đồ dạng cú pháp rút gọn	66
3.4.2	Báo cáo Chứng minh Chuẩn hóa 3NF	67
3.5	Lược đồ Quan hệ Lô-gíc(Logical Schema) & 3NF - Phân hệ minh chứng AI & Giám sát	70
3.5.1	Lược đồ dạng cú pháp rút gọn	72
3.5.2	Báo cáo Chứng minh Chuẩn hóa 3NF	72

4	Tổng kết Lược đồ Quan hệ Lô-gíc-Chuẩn hóa 3NF (Stage 2)	75
4.1	Sơ đồ ERD Mức Logic Hệ thống LivingDocs	77
III	Thiết Kế Vật Lý	79
5	Thiết Kế CSDL Vật Lý & DDL SQL Theo Phân Hệ	81
5.1	Ánh Xạ Lược Đồ ER Phân Hệ Quản Trị Xác Thực & Phân Quyền (Auth & RBAC Domain)	81
5.1.1	Cú pháp Rút gọn (Logical Schema Syntax)	81
5.1.2	Mã DDL SQL Hoàn Chỉnh (<code>schema_member_1.sql</code>)	82
5.2	Ánh Xạ Lược Đồ ER Phân Hệ Không Gian Làm Việc & Tích Hợp VCS (Workspace & External VCS Domain)	86
5.2.1	Cú pháp Rút gọn (Logical Schema Syntax)	86
5.2.2	Mã DDL SQL Hoàn Chỉnh (<code>schema_member_workspace.sql</code>)	87
5.3	Ánh Xạ Lược Đồ ER Phân Hệ Cấu Trúc Mã Nguồn & Phân Tích AST (Code Structure & AST Analysis Domain)	89
5.3.1	Cú pháp Rút gọn (Logical Schema Syntax)	89
5.3.2	Mã DDL SQL Hoàn Chỉnh (<code>schema_member_3.sql</code>)	90
5.4	Ánh Xạ Lược Đồ ER Phân Hệ Khuôn Mẫu & Quản Lý Vòng Đời Tài Liệu (Template & Core Documentation Domain)	96
5.4.1	Cú pháp Rút gọn (Logical Schema Syntax)	96
5.4.2	Mã DDL SQL Hoàn Chỉnh (<code>schema_member_4.sql</code>)	97
5.5	Ánh Xạ Lược Đồ ER Phân Hệ Kiểm Duyệt, Bằng Chứng AI & Giám Sát Hệ Thống (Governance, AI Evidence & Observability Domain)	103
5.5.1	Cú pháp Rút gọn (Logical Schema Syntax)	103
5.5.2	Mã DDL SQL Hoàn Chỉnh (<code>schema_member_5.sql</code>)	103
IV	Tổng Kết	107
6	Tổng Kết	109

Giới Thiệu Đề Tài

Tổng quan đề tài

Trong quy trình phát triển phần mềm hiện đại, việc duy trì tính nhất quán giữa tài liệu kỹ thuật (*Documentation*) và mã nguồn thực tế (*Git Codebase*) là một thách thức lớn. Hiện tượng "**Trôi lệch tài liệu**" (*Documentation Drift*) thường xuyên xảy ra khi mã nguồn liên tục được cập nhật nhưng tài liệu không được điều chỉnh kịp thời, dẫn đến lãng phí thời gian bảo trì và gia tăng rủi ro trong vận hành hệ thống.

Hệ thống **LivingDocs** ra đời như một giải pháp toàn diện giúp tự động hóa việc đồng bộ, kiểm duyệt và quản lý tài liệu kỹ thuật theo vết mã nguồn. Hệ thống tích hợp công nghệ AI (*AI Grounding*) để đối soát minh chứng trực tiếp giữa các đoạn mã nguồn với văn bản tài liệu, phát hiện và cảnh báo kịp thời sự trôi lệch dữ liệu.

Phạm vi Kiến trúc Cơ sở Dữ liệu

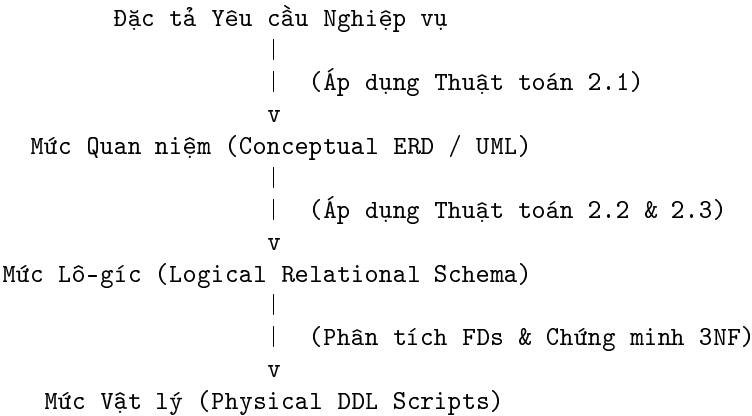
Nhóm thực hiện đề tài xây dựng và thiết kế kiến trúc Cơ sở dữ liệu (CSDL) cho hệ thống **LivingDocs**, bao phủ 5 phân hệ nghiệp vụ nòng cốt:

- **Phân hệ Quản trị & Phân quyền (*Auth & Workspace Domain*):** Quản lý thông tin người dùng (**users**), tổ chức không gian làm việc (**workspaces**), cấu trúc vai trò (**roles**) và chi tiết quyền hạn (**permissions**).
- **Phân hệ Tích hợp Mã nguồn (*Git Repository Domain*):** Đồng bộ cấu trúc dữ liệu từ Git bao gồm kho mã nguồn (**repositories**), nhánh (**branches**), lịch sử thay đổi (**commits**), yêu cầu hợp nhất (**pull_requests**) và các thực thể mã nguồn thu được từ cây AST (**code_entities**).

- **Phân hệ Tài liệu & AI Minh chứng (*Documentation & AI Domain*):** Quản lý dàn ý mẫu (`templates`), các mục dàn ý (`template_sections`), biến chèn (`placeholders`), tài liệu (`documents`), các phiên bản tài liệu (`document_versions`), dữ liệu đối soát AI (`grounding_evidences`) và hệ thống cảnh báo trôi lệch (`drift_alerts`).
- **Phân hệ Kiểm duyệt Kỹ thuật (*Code Review Domain*):** Quản lý các phiên review tài liệu (`reviews`), các quyết định phê duyệt (`approvals`) và luồng bình luận góp ý (`review_comments`).
- **Phân hệ Cấu hình & Nhật ký (*Preferences & Audit Logs*):** Quản lý cấu hình thông báo cá nhân (`user_notification_preferences`), tích hợp kết nối GitHub (`github_connections`) và lưu trữ nhật ký biến động hệ thống (`change_logs`).

Phương pháp Nghiên cứu và Quy trình Triển khai CSDL

Để đảm bảo CSDL của hệ thống LivingDocs đạt tính toàn vẹn, tối ưu hiệu năng và triệt tiêu dư thừa dữ liệu, đồ án áp dụng quy trình thiết kế CSDL chuẩn mực qua 3 mức biểu diễn kết hợp triển khai các thuật toán nền tảng của môn học Thiết kế Cơ sở Dữ liệu:



Phần I

Thiết Kế Quan Niệm

Chương 1

Thuật toán 2.1 & Mô hình Conceptual ERD

Ở mức thiết kế quan niệm (*Conceptual Design Level*), mục tiêu cốt lõi là chuyển hóa các mô tả nghiệp vụ thô, yêu cầu chức năng và quy tắc hệ thống thành mô hình dữ liệu trừu tượng, hoàn toàn độc lập với Hệ quản trị cơ sở dữ liệu (DBMS). Đồ án áp dụng **Thuật toán 2.1** (*Algorithm 2.1: Thiết kế Sơ đồ ER từ Yêu cầu Nghiệp vụ*) để đảm bảo tính toàn vẹn và phản ánh chính xác các nghiệp vụ cốt lõi của hệ thống **LivingDocs — Design and Implementation of an AI-Assisted Self-Updating Document System for Engineering Teams**.

Quá trình này tiếp nhận đầu vào (*Input*) là bài toán giải quyết hiện tượng trôi lệch tài liệu (*Documentation Drift*), bộ yêu cầu chức năng từ 5 vai trò người dùng (*Developer, Staff, Manager, Technical Lead, Administrator*) và kết xuất đầu ra (*Output*) là Sơ đồ Quan niệm (*Conceptual ERD/UML*) đã qua kiểm chứng.

Quy trình Thuật toán 2.1 (*ER Design Algorithm*):

- **Đầu vào (*Input*):** Tài liệu đặc tả nghiệp vụ LivingDocs.
- **Bước 1:** Xác định phạm vi dữ liệu cần lưu trữ (*Data Scope Definition*).
- **Bước 2:** Trích xuất cú pháp đặc tả: Liệt kê danh từ (ứng viên thực thể), động từ (ứng viên mối kết hợp), số lượng và ràng buộc.
- **Bước 3:** Lựa chọn Thực thể, Thuộc tính, Khóa chính (PK) và Mối kết hợp.

4 *CHƯƠNG 1. THUẬT TOÁN 2.1 & MÔ HÌNH CONCEPTUAL ERD*

- **Bước 4:** Gán bản số (*Cardinality*), mức tham gia (*Participation*), xác định các Thực thể yếu (*Weak Entities*) nếu có.
- **Bước 5:** Kiểm thử mô hình bằng dữ liệu mẫu và các kịch bản kiểm thử biên (*Edge Cases*).
- **Đầu ra (*Output*):** Sơ đồ ERD/UML Quan niệm hoàn chỉnh đã qua kiểm chứng.

1.1 Phân tích Nghiệp vụ: Phạm vi, Thực thể & Ràng buộc

1.1.1 Khoanh vùng phạm vi lưu trữ (Scope Boundaries)

Cần lưu

1.1. PHÂN TÍCH NGHIỆP VỤ: PHẠM VI, THỰC THỂ & RÀNG BUỘC⁵

Nhóm	Lý do phải lưu
User, Role, Permission, Workspace	Dữ liệu định danh, RBAC – cốt lõi hệ thống
Repository, GitHubConnection	Cấu hình kết nối, cần cho mọi lần sync
Commit, PullRequest	Là “mốc” để truy vết mọi drift/document version (metadata: sha, number, branch, status)
CodeEntity	Snapshot tại 1 commit: class/method/param. Căn cứ đối chiếu ngữ nghĩa, không thể tính lại nếu code đã đổi
Document, DocumentVersion	Đối tượng nghiệp vụ trung tâm
GroundingEvidence	Source File + Code Entity + Commit Hash. Bắt buộc – chống hallucination, phải truy vết được
DriftEvent	Loại drift, severity. Đầu ra nghiệp vụ chính, cần cho dashboard & báo cáo
ReviewComment	Bằng chứng review 2 cấp (Staff → Manager)
ChangeLog	Actor + Timestamp + Trigger. Yêu cầu NFR: mọi thay đổi phải audit/rollback được
Template, TemplateSection/placeholder mapping	Tái sử dụng nhiều lần, không tính lại

Bảng 1.1: Các nhóm dữ liệu cần lưu

Không lưu

- **Toàn bộ source code:** chỉ lưu path + commit hash + embedding vector (vào vector store riêng), không copy nguyên file vào DB nghiệp vụ.
- **Prompt/response thô của LLM:** chỉ giữ lại grounding evidence (đã

trích) + confidence score; log đầy đủ (nếu cần) đưa vào cold storage/log store riêng, có TTL, không nằm trong schema domain.

- **Access token/refresh token OAuth:** lưu ở kho secret riêng (mã hóa), không phải là một “thực thể nghiệp vụ” trong data dictionary.
- **Số liệu dashboard/health metrics tính realtime:** tính từ dữ liệu gốc lúc query hoặc cache Redis, không phải nguồn sự thật (không cần bảng riêng).
- **Kafka event payload:** là log hạ tầng (event sourcing ngắn hạn), không thuộc schema domain, có retention riêng.
- **Trạng thái UI tạm thời:** tab đang mở, cursor phân trang, ... không lưu.
- **System health/infra metrics:** đẩy sang hệ thống monitoring, không vào DB nghiệp vụ.

1.2 Từ điển nghiệp vụ (Data Dictionary) – chuẩn hóa tên gọi

Tên chuẩn hóa (ENG)	Tên nghiệp vụ (VN)	Mô tả nghiệp vụ
User	Người dùng	Tài khoản cá nhân dùng chung cho mọi vai trò hệ thống.
Role	Vai trò	Nhóm quyền hạn (Developer, Staff, Manager, Tech Lead, Admin).
Permission	Quyền hạn	Chi tiết thao tác được phép thực hiện (VD: DOC_APPROVE, TEMPLATE_EDIT).
Workspace	Không gian làm việc	Môi trường quản lý tập hợp nhiều kho mã nguồn của một đội ngũ/dự án.
GithubConnection	Kết nối GitHub	Thông tin xác thực OAuth, cấp quyền scope và trạng thái đồng bộ tài khoản GitHub.
Repository	Kho mã nguồn	Kho lưu trữ code GitHub/GitLab được đồng bộ vào hệ thống.

1.2. TỪ ĐIỂN NGHIỆP VỤ (DATA DICTIONARY) – CHUẨN HÓA TÊN GỌI

Tên chuẩn hóa (ENG)	Tên nghiệp vụ (VN)	Mô tả nghiệp vụ
Branch	Nhánh mã nguồn	Nhánh phát triển cụ thể trong Repository (VD: main, feature/x).
Commit	Bản ghi Commit	Một mốc thay đổi mã nguồn gắn liền với commit hash.
PullRequest	Yêu cầu hợp nhất (PR)	Đơn vị thay đổi code dùng để kích hoạt tiến trình phân tích drift và tự động cập nhật tài liệu.
CodeDiff	Chi tiết thay đổi code	Chi tiết các dòng code thêm/xóa/sửa trong một Commit hoặc PR.
File	Tập tin mã nguồn	Cấu trúc tệp tin code trong kho chứa.
CodeEntity	Thực thể mã nguồn	Cấu trúc code thu được từ phân tích AST (Class, Method, Endpoint, Parameter, Return value).
Document	Tài liệu gốc	Thực thể quản lý chung cho một tài liệu (README, API Reference, Module Guide, ADR, ...).
DocumentVersion	Phiên bản tài liệu	Bản ghi nội dung tài liệu theo thời gian, quản lý vòng đời (Draft → In Review → Approved → Published).
Template	Mẫu tài liệu	Khung cấu trúc chuẩn dùng để AI hoặc User sinh tài liệu.
TemplateSection	Mục tài liệu mẫu	Chi tiết từng tiêu đề/mục con nằm trong Template.
Placeholder	Vị trí chèn dữ liệu	Điểm biến số trong Section dành để chèn thông tin trích xuất từ mã nguồn.
GroundingEvidence	Minh chứng căn cứ	Liên kết dữ liệu gốc (File + CodeEntity + Commit) giúp đối soát nội dung do AI sinh, chống hallucination.
Drift	Hiện tượng lệch tài liệu	Sự cố tài liệu bị sai lệch/lỗi thời so với mã nguồn thực tế.

Tên chuẩn hóa (ENG)	Tên nghiệp vụ (VN)	Mô tả nghiệp vụ
DriftAlert	Cảnh báo lệch tài liệu	Cảnh báo phát sinh từ PR/Commit khi phát hiện tài liệu bị lỗi thời so với code mới.
Review	Tiến trình kiểm duyệt	Phiên đánh giá kỹ thuật do Staff thực hiện trên một document_version.
ReviewComment	Bình luận review	Góp ý, phản hồi chi tiết của Staff trong quá trình kiểm duyệt.
Notification	Thông báo	Cảnh báo/tin nhắn gửi tới User (khi có PR chờ duyệt, kết nối Token bị hỏng, Drift Alert, ...).
ChangeLog	Nhật ký thay đổi	Lịch sử vết thay đổi tài liệu chi tiết giữa các phiên bản, ghi rõ Actor + Timestamp + Trigger.

Bảng 1.2: Từ điển nghiệp vụ của hệ thống

1.3 Phân loại Tác nhân & Phạm vi Nghiệp vụ (Actor Classification & Business Scope)

1.3.1 Danh mục Tác nhân Hệ thống (System Actors)

Trong mô hình thiết kế quan niệm, hệ thống ghi nhận 5 nhóm **Tác nhân (Actors)**. Tất cả các tác nhân con người đều thuộc thực thể trung tâm User, tác động lên dữ liệu thông qua mô hình phân định vai trò (Role) có phạm vi (Scoped) gắn liền với thực thể Workspace. Một User có thể nắm giữ các vai trò khác nhau trên từng Workspace khác nhau.

Developer

Mục tiêu: người tạo code và là đối tượng chính tiêu thụ + khởi tạo tài liệu.

Quyền hạn chính:

- Quản lý tài khoản: đăng ký/đăng nhập/đăng xuất (email hoặc GitHub OAuth), profile, notification preferences.
- Kết nối/ngắt kết nối GitHub, cấp quyền OAuth, chọn repo vào workspace, trigger sync thủ công.

1.3. PHÂN LOẠI TÁC NHÂN & PHẠM VI NGHĨEP VỤ (ACTOR CLASSIFICATION & BUSINESS SCOP

- Yêu cầu AI sinh tài liệu (theo file/module/repo), chọn template, xem coverage gap, preview/chỉnh sửa tài liệu AI, xem grounding evidence, submit vào review.
- Bật/tắt auto-update tài liệu; xem diff, so sánh version, accept/edit/dismiss/regenerate/rollback.
- Xem drift events liên quan đến PR của mình (loại drift, severity, evidence).
- Dùng Knowledge Base: tìm doc/code entity, xem liên kết Doc ↔ Code.

Phạm vi dữ liệu: chỉ trong workspace/repo mình được cấp quyền; **không** có quyền approve/publish – chỉ submit và chờ Staff/Manager.

Đặc điểm quan trọng: đây là actor duy nhất **khởi tạo** vòng đời tài liệu (bước đầu tiên trong chuỗi AI → Staff → Manager).

Staff

Mục tiêu: review cấp 1 – kiểm tra tính đúng đắn nội dung AI sinh ra trước khi lên Manager.

Quyền hạn chính:

- Xem hàng đợi tài liệu chờ review (queue), filter theo repository / template / severity / submission date.
- Mở tài liệu kèm grounding evidence, source file, code entity, commit, diff, ticket, change log.
- So sánh AI version với approved version; kiểm tra khớp template và khớp source code.
- Comment, yêu cầu chỉnh sửa, sửa lỗi nhỏ trực tiếp.
- Approve (đẩy lên Manager) / Reject / trả lại Developer.

Phạm vi dữ liệu: đọc toàn bộ context kỹ thuật (code, diff, commit) của tài liệu đang review, nhưng không có quyền publish/commit cuối cùng – đó là quyền của Manager.

Đặc điểm quan trọng: là “cổng lọc kỹ thuật” – chặn các tài liệu sai trước khi tốn thời gian Manager.

Manager

Mục tiêu: phê duyệt cuối cùng, chịu trách nhiệm publish/commit vào repository.

Quyền hạn chính:

- Xem tài liệu đã qua Staff review, xem staff comments, grounding evidence, audit trail.
- **Approve để publish/commit, Reject, Rollback.**
- Cấu hình: loại tài liệu nào bắt buộc cần Manager approval, merge policy, kiểm soát publish/commit.
- Xem lịch sử approval; **delegate quyền approval** cho người khác.

Phạm vi dữ liệu: toàn bộ audit trail + kết quả review của Staff, không cần xem lại chi tiết code diff (đã được Staff xử lý), tập trung vào quyết định go/no-go.

Đặc điểm quan trọng: là actor **duy nhất** có quyền publish/commit tài liệu vào repository thật – đúng nguyên tắc “AI không tự ý thay đổi tài liệu mà không kiểm soát”.

Technical Lead

Mục tiêu: quản trị (governance) toàn bộ hệ thống tài liệu ở tầm chiến lược, không tham gia review từng tài liệu đơn lẻ.

Quyền hạn chính:

- Quản lý documentation collection, phân quyền ownership.
- Cấu hình **drift detection policy** và **threshold**.
- Xem documentation health dashboard, dependency graph, quality metrics, theo dõi unresolved drift, xuất báo cáo.
- Cấu hình auto-update (repo nào, module nào, loại tài liệu nào), xem version history/change log, rollback ở cấp hệ thống.
- Quản lý Template: tạo/sửa/clone/xóa, định nghĩa section/heading/placeholder, mapping placeholder ↔ code entity.

1.3. PHÂN LOẠI TÁC NHÂN & PHẠM VI NGHIỆP VỤ (ACTOR CLASSIFICATION & BUSINESS SCOPES)

Phạm vi dữ liệu: toàn bộ workspace/repo thuộc phạm vi quản lý – nhìn theo **chỉ số tổng hợp** (dashboard, metrics) hơn là từng tài liệu.

Đặc điểm quan trọng: khác biệt với Manager ở chỗ Manager phê duyệt **từng tài liệu**, còn Tech Lead cấu hình **chính sách/threshold áp dụng cho toàn hệ thống** (drift policy, template chuẩn).

Administrator

Mục tiêu: vận hành hạ tầng & tích hợp toàn hệ thống.

Quyền hạn chính: quản lý user, role, permission, workspace, repository; cấu hình GitHub App/OAuth/GitHub Actions; tích hợp GitLab/Jira/Slack; cấu hình supported programming languages; cấu hình AI settings (LLM, embedding model, confidence threshold, token budget, indexing, retrieval); notification; theo dõi system health, AI processing jobs, event pipeline, audit log, system analytics.

Phạm vi dữ liệu: toàn hệ thống, cấp hạ tầng/kỹ thuật – **không** liên quan trực tiếp đến nội dung nghiệp vụ tài liệu (không review/approve tài liệu).

Đặc điểm quan trọng: là actor duy nhất chạm vào **cấu hình AI model** (LLM, embedding, threshold) – tách biệt hoàn toàn khỏi luồng nghiệp vụ review/approval.

1.3.2 Chi tiết các Thực thể Dữ liệu Cốt lõi (Core Data Entities)

Nhóm Quản trị & Kết nối

- **User (Thực thể mạnh):** Người dùng cuối trong hệ thống (Developer, Staff, Manager, Tech Lead, Admin).
- **Workspace (Thực thể mạnh):** Không gian làm việc chứa repository và tài liệu của một nhóm/dự án.
- **Role (Thực thể mạnh):** Vai trò của người dùng (scoped theo Workspace).
- **Permission (Thực thể mạnh):** Quyền hạn chi tiết trên hệ thống (VD: DOC_APPROVE, TEMPLATE_EDIT).
- **GitHubConnection (Thực thể phụ thuộc):** Lưu thông tin xác thực OAuth của User với GitHub.

Nhóm Mã nguồn & Cấu trúc Code

- **Repository (Thực thể mạnh):** Kho chứa mã nguồn được đồng bộ vào hệ thống.
- **Branch (Thực thể yếu – phụ thuộc Repository):** Nhánh phát triển trong repository.
- **Commit (Thực thể yếu – phụ thuộc Branch/Repo):** Bản ghi một lần commit mã nguồn.
- **PullRequest (Thực thể yếu – phụ thuộc Repository):** Yêu cầu hợp nhất nhánh (PR) từ GitHub.
- **CodeDiff (Thực thể yếu – phụ thuộc Commit/PR):** Chi tiết thay đổi mã nguồn trong commit hoặc PR.
- **File (Thực thể yếu – phụ thuộc Repository):** Cấu trúc tập tin mã nguồn trong kho chứa.
- **CodeEntity (Thực thể yếu – phụ thuộc File):** Đơn vị mã nguồn cụ thể thu được từ AST (Class, Method, Function, ...).
- **Dependency (Thực thể mối kết hợp):** Mối quan hệ phụ thuộc giữa các CodeEntity.

Nhóm Khuôn mẫu Tài liệu (Template Engine Domain)

- **Template (Thực thể mạnh):** Khuôn mẫu chuẩn để AI/User sinh tài liệu.
- **TemplateSection (Thực thể yếu – phụ thuộc Template):** Mục con/heading trong cấu trúc dàn ý của Template.
- **Placeholder (Thực thể yếu – phụ thuộc TemplateSection):** Biến/điểm chèn trong Section, làm nhiệm vụ nhận dữ liệu từ code.

Nhóm Tài liệu & Vòng đời Dữ liệu (Documentation Domain)

- **Document (Thực thể mạnh):** Tài liệu gốc (README, API Reference, Architecture Guide, ADR, ...).
- **DocumentVersion (Thực thể yếu – phụ thuộc Documentation):** Bản ghi nội dung tài liệu theo từng mốc thời gian và trạng thái (Draft, In Review, Approved, Published).

- **ChangeLog** (Thực thể yếu – phụ thuộc **DocumentVersion**): Lịch sử vết thay đổi chi tiết giữa các phiên bản tài liệu.

Nhóm Kiểm duyệt & Phê duyệt (Review & Governance Domain)

- **Review** (Thực thể yếu-phụ thuộc **DocumentVersion**): Tiến trình Staff thực hiện đánh giá kỹ thuật trên một **DocumentVersion**.
- **ReviewComment** (Thực thể yếu – phụ thuộc **Review**): Phản hồi, góp ý chi tiết trong quá trình kiểm duyệt.

Nhóm AI Evidence, Giám sát & Kiểm toán (AI Evidence & Observability Domain)

- **GroundingEvidence** (Thực thể yếu – phụ thuộc **DocumentVersion**): Bảng minh chứng chứa liên kết nguồn gốc (File, CodeEntity, Commit) AI dùng để tạo nội dung, phục vụ chống hallucination.
- **DriftAlert** (Thực thể sự kiện): Cảnh báo phát hiện tài liệu bị lệch ngữ nghĩa/lỗi thời so với code mới.
- **Approval** (Thực thể mối kết hợp / Sự kiện): Quyết định phê duyệt xuất bản hoặc *commit* tài liệu chính thức từ phía Manager.

“ “

1.4 Chi tiết các Thực thể Dữ liệu

1.4.1 Nhóm Quản trị & Kết nối

User (Thực thể mạnh)

Người dùng cuối trong hệ thống (Developer, Staff, Manager, Tech Lead, Admin).

Thuộc tính Phức hợp (Composite Attribute)

- **FullName**: Thuộc tính phức hợp gồm:
 - **FirstName**
 - **LastName**

Thuộc tính Đa trị (Multi-valued Attribute)

- **NotificationPreferences (Cài đặt thông báo):** Lưu nhiều giá trị cùng lúc cho một người dùng (ví dụ: chọn đồng thời cả **EMAIL** và **SYSTEM**).

Thuộc tính Đơn (Simple / Atomic Attribute)

- **user_id (PK):UUID** Định danh duy nhất cho từng người dùng.
- **email:** Địa chỉ email bắt buộc dùng để đăng nhập và nhận thông báo hệ thống.
- **password_hash:** Mật khẩu đã mã hóa. Lưu giá trị khi đăng nhập bằng Email/Password và giữ **NULL** khi dùng thuần GitHub OAuth.
- **status:** Trạng thái tài khoản (**ACTIVE**, **INACTIVE**, **BLOCKED**).
- **created_at:** Thời điểm khởi tạo tài khoản.
- **updated_at:** Thời điểm cập nhật thông tin gần nhất.
- **deleted_at:** Soft delete nhằm giữ nguyên lịch sử giao dịch (review, comment, approval) khi người dùng rời hệ thống.

1.4.2 Workspace (Thực thể mạnh)

Không gian làm việc chứa repository và tài liệu của một nhóm/dự án.

Thuộc tính Đơn (Simple / Atomic Attribute)

- **workspace_id (PK):UUID** Định danh duy nhất cho từng không gian làm việc.
- **name:** Tên không gian làm việc/dự án (VD: “Payment Service Platform”).
- **description:** Mô tả chi tiết về phạm vi và mục tiêu của workspace.
- **status:** Trạng thái hoạt động của workspace (**ACTIVE**, **ARCHIVED**).
- **created_at:** Thời điểm khởi tạo workspace.
- **updated_at:** Thời điểm cập nhật thông tin workspace gần nhất.
- **deleted_at:** Cờ xóa mềm (Soft delete) giữ nguyên dữ liệu báo cáo và audit trail khi ẩn workspace.

1.4.3 Role (Thực thể mạnh)

Vai trò của người dùng, được scoped theo Workspace.

Thuộc tính Đơn (Simple / Atomic Attribute)

- **role_id (PK):UUID** Định danh duy nhất cho vai trò.
- **role_name:** Tên vai trò (VD: Developer, Staff, Manager, Tech Lead, Admin).
- **description:** Mô tả tóm tắt mục đích và quyền hạn chung của vai trò.
- **created_at:** Thời điểm khởi tạo vai trò trong hệ thống.
- **updated_at:** Thời điểm cập nhật vai trò gần nhất.

Permission (Thực thể mạnh)

Quyền hạn chi tiết trên hệ thống (VD: DOC_APPROVE, TEMPLATE_EDIT).

Thuộc tính Đơn (Atomic Attribute)

- **permission_id (PK):UUID** Định danh duy nhất cho quyền.
- **permission_code:** Mã quyền thao tác hệ thống (VD: DOC_APPROVE, TEMPLATE_EDIT).
- **description:** Giải thích chi tiết giới hạn của thao tác này.
- **created_at:** Mốc thời gian quyền được khởi tạo vào danh mục hệ thống.

RolePermission (Thực thể liên kết)

Bản ghi phân quyền chi tiết cho từng vai trò trong hệ thống.

Thuộc tính Đơn (Atomic Attribute)

- **role_permission_id (PK):UUID** Định danh duy nhất cho bản ghi phân quyền (UUID).
- **role_id (FK):UUID** Khóa ngoại trỏ đến vai trò được gán quyền (roles).

- **permission_id (FK):UUID** Khóa ngoại trỏ đến quyền hạn cụ thể (permissions).
- **created_at**: Mốc thời gian quyền này được gán cho vai trò.

WorkspaceMember (Thực thể mối kết hợp)

Ghi nhận sự tham gia của User trong Workspace và vai trò (Role) tương ứng.

Thuộc tính Đơn (Atomic Attribute)

- **member_id (PK):UUID** Định danh duy nhất cho bản ghi tham gia.
- **status**: Trạng thái thành viên (ACTIVE, INACTIVE, REMOVED).
- **joined_at**: Thời điểm người dùng chính thức tham gia.
- **created_at**: Thời điểm gửi lời mời / tạo bản ghi.
- **updated_at**: Thời điểm cập nhật vai trò hoặc trạng thái gần nhất.

GitHubConnection (Thực thể phụ thuộc)

Lưu thông tin xác thực OAuth của User với GitHub.

Thuộc tính Khóa (Key Attribute)

- **connection_id**: Định danh duy nhất cho bản ghi kết nối OAuth.

Thuộc tính Đơn / Nguyên tố (Simple / Atomic Attributes)

- **github_user_id**: ID tài khoản người dùng trên hệ thống GitHub để đối soát định danh.
- **github_username**: Tên tài khoản GitHub (VD: octocat).
- **access_token**: Chuỗi token ủy quyền dùng để gọi GitHub API.
- **created_at**: Thời điểm người dùng thực hiện liên kết tài khoản.
- **updated_at**: Thời điểm làm mới token hoặc cập nhật quyền hạn gần nhất.

Thuộc tính Đa trị (Multi-valued Attribute)

- **scopes:** Lưu danh sách các phạm vi quyền OAuth được cấp phép cùng lúc (VD: repo, read:user, workflow).

1.4.4 Nhóm Mã nguồn & Cấu trúc Code**Repository (Thực thể mạnh)**

Kho chứa mã nguồn được đồng bộ vào hệ thống.

Thuộc tính Đơn (Atomic Attribute)

- **repository_id (PK):**UUID Định danh duy nhất cho kho mã nguồn.
- **name:** Tên kho mã nguồn (VD: payment-gateway).
- **url:** Đường dẫn gốc đến kho chứa trên VCS.
- **default_branch:** Nhánh mặc định dùng để phân tích code (VD: main, master).
- **sync_status:** Trạng thái đồng bộ dữ liệu (SYNCING, SUCCESS, FAILED).
- **last_synced_at:** Thời điểm đồng bộ mã nguồn thành công gần nhất.
- **created_at:** Thời điểm tích hợp kho mã nguồn vào hệ thống.
- **updated_at:** Thời điểm cập nhật trạng thái hoặc cấu hình kho gần nhất.

Branch (Thực thể yếu – phụ thuộc Repository)

Nhánh phát triển trong repository.

Thuộc tính Đơn (Simple / Atomic Attribute)

- **branch_id (PK):**UUID Định danh duy nhất cho nhánh.
- **branch_name:** Tên nhánh (VD: main, staging, feature/auth).
- **is_default:** Cờ xác định nhánh mặc định của repo (true/false).
- **created_at:** Thời điểm nhánh được nhận diện và lưu vào hệ thống.
- **updated_at:** Thời điểm cập nhật commit hash hoặc trạng thái nhánh gần nhất.

Commit (Thực thể yếu – phụ thuộc Branch/Repository)

Bản ghi một lần commit mã nguồn.

Thuộc tính Phức hợp (Composite Attribute)

- **AuthorInfo:** Thông tin tác giả thực hiện commit, gồm:
 - AuthorName
 - AuthorEmail

Thuộc tính Đơn (Simple / Atomic Attribute)

- **commit_id (PK):UUID** Định danh nội bộ cho bản ghi commit.
- **commit_hash:** Mã băm SHA-1/SHA-256 duy nhất từ Git (VD: 7a8f9b2...).
- **message:** Nội dung mô tả của commit.
- **committed_at:** Thời điểm commit được thực hiện trên VCS (thời gian nghiệp vụ).
- **created_at:** Thời điểm bản ghi commit được đồng bộ vào LivingDocs (thời gian hệ thống).

PullRequest (Thực thể yếu – phụ thuộc Repository)

Yêu cầu hợp nhất nhánh từ GitHub.

Thuộc tính Đa trị (Multi-valued Attribute)

- **labels:** Danh sách nhãn phân loại được gắn trên PR (VD: bug, enhancement, documentation, api-change).

Thuộc tính Đơn (Simple / Atomic Attribute)

- **pr_id (PK):UUID** Định danh nội bộ của bản ghi PR.
- **pr_number:** Số thứ tự PR trên nền tảng gốc (VD: #102, #123).
- **title:** Tiêu đề của Pull Request.
- **description:** Nội dung văn bản mô tả chi tiết PR, là dữ liệu đầu vào cho AI.

- **source_branch:** Nhánh nguồn chứa mã nguồn thay đổi.
- **target_branch:** Nhánh đích dự định hợp nhất vào (VD: main, develop).
- **status:** Trạng thái vòng đời của PR (OPEN, MERGED, CLOSED).
- **created_at:** Thời điểm mở PR trên VCS.
- **updated_at:** Thời điểm PR cập nhật thông tin/trạng thái gần nhất.
- **merged_at:** Thời điểm PR chính thức được chấp nhận hợp nhất (Nullable).

CodeDiff (Thực thể yếu – phụ thuộc Commit/PR)

Chi tiết thay đổi mã nguồn trong commit hoặc PR.

Thuộc tính Đơn (Simple / Atomic Attribute)

- **diff_id (PK):**UUID Định danh nội bộ bản ghi chi tiết thay đổi.
- **file_path:** Đường dẫn tương đối đến tệp bị thay đổi (VD: src/controllers/payment.js).
- **change_type:** Loại thao tác trên file (ADD, MODIFY, DELETE, RENAME).
- **additions:** Số dòng code được thêm mới.
- **deletions:** Số dòng code bị xóa đi.
- **diff_content:** Nội dung chi tiết các đoạn code thay đổi (patch/hunk text) dùng làm input cho AI.
- **created_at:** Thời điểm lưu bản ghi diff vào hệ thống.

File (Thực thể yếu – phụ thuộc Repository)

Cấu trúc tập tin mã nguồn trong kho chứa.

Thuộc tính Đơn (Simple / Atomic Attribute)

- **file_id (PK):UUID** Định danh nội bộ của tập tin trong hệ thống.
- **file_path**: Đường dẫn đầy đủ của file tính từ gốc repository.
- **file_type**: Loại hoặc đuôi mở rộng của tệp (VD: ts, java, py, md).
- **status**: Trạng thái của file trong cây thư mục (ACTIVE, DELETED, RENAMED).
- **created_at**: Thời điểm file lần đầu được ghi nhận vào hệ thống.
- **updated_at**: Thời điểm thông tin file hoặc nội dung file thay đổi gần nhất.

CodeEntity (Thực thể yếu – phụ thuộc File)

Đơn vị mã nguồn cụ thể thu được từ AST (Class, Method, Function...).

Thuộc tính Phức hợp (Composite Attribute)

- **position**: Vị trí của thực thể trong tập tin, gồm:
 - **start_line**: Dòng bắt đầu của thực thể code.
 - **end_line**: Dòng kết thúc của thực thể code.

Thuộc tính Đơn (Simple / Atomic Attribute)

- **entity_id (PK):UUID** Định danh nội bộ của thực thể code.
- **entity_name**: Tên của thành phần code (VD: processPayment, UserController).
- **entity_type**: Loại thành phần code (CLASS, METHOD, FUNCTION, INTERFACE, ENDPOINT).
- **signature**: Cấu trúc chữ ký của thành phần code.
- **doc_comment**: Đoạn comment/docstring nguyên bản đính kèm hàm/lớp (nếu có).
- **created_at**: Thời điểm thực thể code được bóc tách và lưu vào hệ thống.
- **updated_at**: Thời điểm cấu trúc của thực thể code thay đổi gần nhất.

Dependency (Thực thể mối kết hợp)

Mối quan hệ phụ thuộc giữa các CodeEntity.

Thuộc tính Đơn (Simple / Atomic Attribute)

- **dependency_id (PK):**UUID Định danh duy nhất cho bản ghi phụ thuộc.
- **dependency_type:** Loại quan hệ phụ thuộc (CALLS, INHERITS, IMPLEMENTS, USES, IMPORTS).
- **created_at:** Thời điểm hệ thống phân tích AST và ghi nhận mối quan hệ phụ thuộc.

1.4.5 Nhóm Khuôn mẫu Tài liệu (Template Engine Domain)**Template (Thực thể mạnh)**

Khuôn mẫu chuẩn để AI/User sinh tài liệu.

Thuộc tính Đơn (Simple / Atomic Attribute)

- **template_id (PK):**UUID Định danh duy nhất cho khuôn mẫu tài liệu.
- **name:** Tên khuôn mẫu.
- **description:** Mô tả mục đích sử dụng và phạm vi áp dụng.
- **content_structure:** Nội dung cấu trúc dàn ý mẫu/Prompt Mark-down hướng dẫn AI và User.
- **creator_type:** Đối tượng khởi tạo khuôn mẫu (SYSTEM, USER, AI).
- **status:** Trạng thái sử dụng (DRAFT, ACTIVE, ARCHIVED).
- **created_at:** Thời điểm tạo khuôn mẫu.
- **updated_at:** Thời điểm cập nhật dàn ý/nội dung khuôn mẫu gần nhất.

TemplateSection (Thực thể yếu – phụ thuộc Template)

Mục con/heading trong cấu trúc dàn ý của Template.

- **section_id (PK):UUID** Định danh duy nhất cho mục dàn ý.
- **heading_title:** Tiêu đề của mục.
- **order_index:** Thứ tự sắp xếp của mục trong Template.
- **prompt_instruction:** Câu lệnh hướng dẫn riêng cho AI khi sinh nội dung cho mục này.
- **created_at:** Thời điểm khởi tạo mục dàn ý.
- **updated_at:** Thời điểm cập nhật tiêu đề hoặc chỉ thị gần nhất.

Placeholder (Thực thể yếu – phụ thuộc TemplateSection)

Biến/điểm chèn trong Section, làm nhiệm vụ nhận dữ liệu từ code.

- **placeholder_id (PK):UUID** Định danh duy nhất cho điểm chèn biến.
- **variable_name:** Mã nhận diện biến.
- **expected_data_type:** Loại dữ liệu kỳ vọng từ code.
- **description:** Mô tả ý nghĩa và ngữ cảnh dữ liệu cần điền vào biến.
- **is_required:** Cờ xác định biến này bắt buộc AI phải lấp đầy hay không.
- **created_at:** Thời điểm tạo điểm chèn biến.
- **updated_at:** Thời điểm cập nhật thông tin biến gần nhất.

TemplateMapping (Thực thể mối kết hợp)

Ánh xạ trực tiếp giữa Placeholder và CodeEntity để AI trích xuất thông tin.

- **mapping_id (PK):UUID** Định danh duy nhất cho bản ghi ánh xạ trích xuất.
- **extraction_rule:** Quy tắc/Prompt chỉ thị chi tiết để AI biết cách lọc, biến đổi và trích xuất dữ liệu từ CodeEntity.

- **is_active:** Trạng thái hiệu lực của quy tắc ánh xạ.
- **created_at:** Thời điểm khởi tạo quy tắc ánh xạ.
- **updated_at:** Thời điểm cập nhật quy tắc trích xuất gần nhất.

1.4.6 Nhóm Tài liệu & Vòng đời Dữ liệu (Documentation Domain)

Document (Thực thể mạnh)

Tài liệu gốc (README, API Reference, Architecture Guide, ADR...).

- **document_id (PK):UUID** Định danh duy nhất cho tài liệu chính thức.
- **title:** Tên/Tiêu đề của tài liệu.
- **document_type:** Phân loại loại tài liệu (README, API_REFERENCE, ARCHITECTURE_GUIDE, ADR, CUSTOM).
- **status:** Trạng thái vòng đời của tài liệu (PUBLISHED, ARCHIVED, DEPRECATED).
- **created_at:** Thời điểm khởi tạo tài liệu.
- **updated_at:** Thời điểm cập nhật tài liệu gần nhất.
- **deleted_at:** Thời điểm xóa mềm tài liệu (Nullable).

DocumentVersion (Thực thể yếu – phụ thuộc Document)

Bản ghi nội dung tài liệu theo từng mốc thời gian và trạng thái.

- **version_id (PK):UUID** Định danh duy nhất cho bản ghi phiên bản.
- **version_number:** Số hiệu phiên bản tài liệu (VD: 1.0.0, 1.1.0, 2.0-draft).
- **content:** Toàn bộ nội dung văn bản tài liệu dạng Markdown/HTML tại thời điểm chụp phiên bản.
- **change_summary:** Tóm tắt mô tả lý do hoặc nội dung những điểm thay đổi trong phiên bản này.

- **status:** Trạng thái vòng đời phiên bản (DRAFT, IN_REVIEW, APPROVED, PUBLISHED).
- **ticket_codes:** Danh sách các mã ticket/thẻ công việc liên quan từ các hệ thống bên ngoài như Jira Ticket ID hoặc GitHub Issue ID. Thuộc tính này được lưu dưới dạng chuỗi văn bản đơn giản, Nullable và phân cách bằng dấu phẩy.
- **created_at:** Thời điểm khởi tạo bản ghi phiên bản.
- **updated_at:** Thời điểm cập nhật trạng thái phiên bản gần nhất.

ChangeLog (Thực thể yếu – phụ thuộc DocumentVersion)

Lịch sử vết thay đổi chi tiết giữa các phiên bản tài liệu.

- **log_id (PK):UUID** Định danh duy nhất cho bản ghi nhật ký thay đổi.
- **change_type:** Loại thao tác biến đổi (ADDED, MODIFIED, DELETED, RESTORED).
- **change_description:** Mô tả tóm tắt ý nghĩa sự thay đổi.
- **diff_content:** Nội dung chênh lệch chi tiết (dạng Git patch / JSON delta).
- **created_at:** Thời điểm ghi nhận vết thay đổi.
- **updated_at:** Thời điểm cập nhật bản ghi nhật ký nếu có bổ sung metadata.

1.4.7 Nhóm Kiểm duyệt & Phê duyệt (Review & Governance Domain)

Review (Thực thể mối kết hợp/Sự kiện)

Tiến trình Staff thực hiện đánh giá kỹ thuật trên một DocumentVersion.

- **review_id (PK):UUID** Định danh duy nhất cho phiên đánh giá kỹ thuật.
- **status:** Trạng thái của tiến trình review (PENDING, IN_PROGRESS, COMPLETED).

- **decision:** Kết quả thẩm định của reviewer (APPROVED, REJECTED, CHANGES_REQUESTED).
- **summary_notes:** Nhận xét hoặc ghi chú tổng quan của reviewer (Nullable).
- **created_at:** Thời điểm bắt đầu phiên đánh giá.
- **updated_at:** Thời điểm chốt kết quả hoặc cập nhật tiến trình review.

ReviewComment (Thực thể yếu – phụ thuộc Review)

Phản hồi, góp ý chi tiết trong quá trình kiểm duyệt.

- **comment_id (PK):UUID** Định danh duy nhất cho bản ghi bình luận/góp ý.
- **content:** Nội dung chi tiết của lời nhận xét, phản hồi.
- **is_resolved:** Trạng thái xử lý góp ý (false: Chưa sửa / true: Đã sửa xong).
- **created_at:** Thời điểm gửi bình luận.
- **updated_at:** Thời điểm cập nhật nội dung bình luận gần nhất.

Approval (Thực thể mối kết hợp/Sự kiện)

Quyết định phê duyệt xuất bản/commit tài liệu chính thức từ Manager.

- **approval_id (PK):UUID** Định danh duy nhất cho bản ghi quyết định phê duyệt.
- **decision:** Trạng thái/Kết quả quyết định (APPROVED, REJECTED).
- **rejection_reason:** Lý do từ chối xuất bản (Nullable, dùng khi decision = REJECTED).
- **approved_at:** Mốc thời gian ra quyết định phê duyệt hoặc từ chối.

1.4.8 Nhóm AI Evidence, Giám sát & Kiểm toán

GroundingEvidence (Thực thể yếu – phụ thuộc DocumentVersion)

Bảng minh chứng chứa liên kết nguồn gốc (File, CodeEntity, Commit) AI dùng để tạo nội dung, phục vụ chống hallucination.

- **evidence_id (PK):UUID** Định danh duy nhất cho bản ghi bằng chứng minh chứng.
- **snippet_content**: Đoạn mã nguồn hoặc văn bản thô trích xuất từ code làm căn cứ trực tiếp cho AI sinh nội dung.
- **purpose**: Phân loại mục đích lưu vết (HALLUCINATION_PREVENTION, CODE_REFERENCE, AST_VERIFICATION).
- **created_at**: Thời điểm ghi nhận bản ghi bằng chứng.

DriftAlert (Thực thể sự kiện)

Cảnh báo phát hiện tài liệu bị lệch ngữ nghĩa/lỗi thời so với code mới.

Thuộc tính Khóa (Key Attribute)

- **alert_id**: Định danh duy nhất cho từng bản ghi cảnh báo.

Thuộc tính Đơn / Nguyên tố (Simple / Atomic Attributes)

- **drift_type**: Phân loại hình thức lệch pha (SEMANTIC, OUTDATED).
- **status**: Trạng thái xử lý cảnh báo (OPEN, RESOLVED, IGNORED).
- **detected_at**: Thời điểm hệ thống AI quét và phát hiện ra độ lệch.

1.4.9 Thực thể mối kết hợp / Sự kiện: Approval

- **Tên thực thể**: Approval (*Phê duyệt*)
- **Mô tả nghiệp vụ**: Quyết định phê duyệt xuất bản hoặc *commit* tài liệu chính thức từ phía Manager.
- **Thuộc tính đơn & Khóa (Atomic & Key Attributes)**:
 - **approval_id (PK)**: Định danh duy nhất cho bản ghi quyết định phê duyệt (UUID).

- **version_id (FK)**: Khóa ngoại trỏ đến phiên bản tài liệu (`document_versions`) được xét duyệt.
- **user_id (FK)**: Khóa ngoại trỏ đến Manager (`users`) thực hiện ra quyết định.
- **decision**: Trạng thái/Kết quả quyết định (`APPROVED`, `REJECTED`).
- **rejection_reason**: Lý do từ chối xuất bản (*Nullable*, dùng khi `decision = REJECTED`).
- **approved_at**: Mốc thời gian ra quyết định phê duyệt hoặc từ chối (`DATETIME2` / `TIMESTAMP`).

1.5 Mô hình hóa Mối kết hợp & Bản số Dữ liệu

1.5.1 Nhóm Quản trị & Kết nối (Auth & Workspace Domain)

1.5.2 Mối kết hợp 3 ngôi: User — Workspace — Role

- **Tên mối kết hợp**: `<participates>`
- **Các thực thể tham gia**: User (Thực thể mạnh), Workspace (Thực thể mạnh), Role (Thực thể mạnh)
- **Tỷ lệ bản số tổng quát**:
- **Bản số chi tiết (Min..Max)**:
 - User: (0..*)
 - Workspace: (1..*)
 - Role: (0..*)
- **Mức tham gia**:
 - User: Tùy chọn (Partial)
 - Workspace: Bắt buộc (Total)
 - Role: Tùy chọn (Partial)
- **Thuộc tính mối quan hệ**: `joined_at`, `status`.

- **Mô tả nghiệp vụ:** Ghi nhận sự tham gia của Người dùng vào một Không gian làm việc. Trong phạm vi từng Workspace cụ thể, mỗi người dùng bắt buộc được gán đúng một Vai trò xác định (Scoped RBAC). Một người dùng có thể tham gia nhiều Workspace khác nhau với các vai trò độc lập (ví dụ: là Manager ở Workspace A nhưng là Developer ở Workspace B).

1.5.3 Mỗi quan hệ: **Role — contains — Permission**

- **Phân loại thực thể:** Role (Thực thể mạnh) $\leftrightarrow$ Permission (Thực thể mạnh)
- **Tên mối kết hợp:** <contains>
- **Tỷ lệ bản số:** M - N
- **Bản số chi tiết:** Role (0..*) $\leftrightarrow$ Permission (0..*)
- **Mức tham gia:** Role: Tùy chọn (Partial) | Permission: Tùy chọn (Partial)
- **Thuộc tính mối kết hợp:** granted_at.
- **Mô tả nghiệp vụ:** Một vai trò chứa tập hợp nhiều quyền; một quyền hạn có thể thuộc về nhiều vai trò khác nhau.

1.5.4 Mỗi quan hệ: **User — connects — Github_Connection**

- **Phân loại thực thể:** User (Thực thể mạnh) $\leftrightarrow$ Github_Connection (Thực thể phụ thuộc – phụ thuộc User)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:**
 - User: (0..*)
 - Github_Connection: (1..1)
- **Mức tham gia:**
 - User: Tùy chọn (Partial)
 - Github_Connection: Bắt buộc (Total)

- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Mối quan hệ ghi nhận việc Người dùng (User) ủy quyền và liên kết tài khoản cá nhân với hệ thống GitHub thông qua chuẩn OAuth2. Kết nối này lưu trữ `access_token` và danh sách các scopes được cấp phép nhằm phục vụ việc đồng bộ repository, đọc commit, Pull Request và cấu hình webhook cho hệ thống.

1.5.5 Nhóm Mã nguồn & Cấu trúc Code (VCS & Code Domain)

1.5.6 Mối quan hệ: **Workspace** — owns — **Repository**

- **Phân loại thực thể:** Workspace (Thực thể mạnh) ↔ Repository (Thực thể mạnh)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** Workspace (1..1) ↔ Repository (0..*)
- **Mức tham gia:** Workspace: Tùy chọn (Partial) | Repository: Bắt buộc (Total)
- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một Workspace quản lý nhiều Repository; mỗi Repository thuộc quyền sở hữu của một Workspace.

1.5.7 Mối quan hệ: **Repository** — has_branch — **Branch**

- **Phân loại thực thể:** Repository (Thực thể mạnh) ↔ Branch (Thực thể yếu – phụ thuộc Repository)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** Repository (1..1) ↔ Branch (1..*)
- **Mức tham gia:** Repository: Bắt buộc (Total) | Branch: Bắt buộc (Total)
- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một kho chứa mã nguồn có ít nhất một nhánh mặc định; mỗi nhánh bắt buộc nằm trong duy nhất một kho chứa.

1.5.8 Mỗi quan hệ: Branch — contains_commit — Commit

- **Phân loại thực thể:** Branch (Thực thể yếu) $\leftrightarrow$ Commit (Thực thể yếu – phụ thuộc Branch/Repository)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** Branch (0..*) $\leftrightarrow$ Commit (1..1)
- **Mức tham gia:** Branch: Tùy chọn (Partial) | Commit: Bắt buộc (Total)
- **Thuộc tính mỗi quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một nhánh chứa nhiều lượt commit; mỗi commit thuộc về đúng một nhánh phát triển.

1.5.9 Mỗi quan hệ: User — makes_commit — Commit

- **Phân loại thực thể:** User (Thực thể mạnh) $\leftrightarrow$ Commit (Thực thể yếu)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** User (0..*) $\leftrightarrow$ Commit (1..1)
- **Mức tham gia:** User: Tùy chọn (Partial) | Commit: Bắt buộc (Total)
- **Thuộc tính mỗi quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một người dùng thực hiện nhiều commit; mỗi commit được tạo bởi đúng một người dùng.

1.5.10 Mỗi quan hệ: Repository — receives_pr — PullRequest

- **Phân loại thực thể:** Repository (Thực thể mạnh) $\leftrightarrow$ PullRequest (Thực thể yếu – phụ thuộc Repository)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** Repository (0..*) $\leftrightarrow$ PullRequest (1..1)
- **Mức tham gia:** Repository: Tùy chọn (Partial) | PullRequest: Bắt buộc (Total)

- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một Repository chứa nhiều Yêu cầu hợp nhất (PR); mỗi PR gửi về đúng một kho chứa.

1.5.11 Mối quan hệ: Commit — generates_diff — CodeDiff

- **Phân loại thực thể:** Commit (Thực thể yếu) ↔ CodeDiff (Thực thể yếu – phụ thuộc Commit/PullRequest)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** Commit (1..*) ↔ CodeDiff (1..1)
- **Mức tham gia:** Commit: Bắt buộc (Total) | CodeDiff: Bắt buộc (Total)
- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một Commit sinh ra một hoặc nhiều bản ghi khác biệt code (Diff); mỗi bản ghi diff thuộc về duy nhất một commit.

1.5.12 Mối quan hệ: PullRequest — includes_commit — Commit

- **Phân loại thực thể:** PullRequest (Thực thể yếu) ↔ Commit (Thực thể yếu)
- **Tỷ lệ bản số:**
- **Bản số chi tiết:** PullRequest (1..*) ↔ Commit (1..1)
- **Mức tham gia:** PullRequest: Bắt buộc (Total) | Commit: Tùy chọn (Partial)
- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một Yêu cầu hợp nhất (PR) tổng hợp danh sách gồm một hoặc nhiều commit; mỗi commit trong luồng xử lý thuộc về đúng một PR.

1.5.13 Mỗi quan hệ: Commit — modifies — File

- **Phân loại thực thể:** Commit (Thực thể yếu) ↔ File (Thực thể yếu – phụ thuộc Repository)
- **Tên mỗi kết hợp:** <modifies>
- **Tỷ lệ bản số:** M - N
- **Bản số chi tiết:** Commit (1..*) ↔ File (0..*)
- **Mức tham gia:** Commit: Bắt buộc (Total) | File: Tùy chọn (Partial)
- **Thuộc tính mỗi quan hệ:** change_type, lines_added, lines_deleted.
- **Mô tả nghiệp vụ:** Một Commit tác động chỉnh sửa ít nhất một File; một File có thể chịu sự biến đổi qua nhiều lượt Commit.

1.5.14 Mỗi quan hệ: Repository — contains_file — File

- **Phân loại thực thể:** Repository (Thực thể mạnh) ↔ File (Thực thể yếu – phụ thuộc Repository)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** Repository (0..*) ↔ File (1..1)
- **Mức tham gia:** Repository: Tùy chọn (Partial) | File: Bắt buộc (Total)
- **Thuộc tính mỗi quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một Repository quản lý cấu trúc cây thư mục chứa nhiều File mã nguồn; mỗi File thuộc một kho chứa duy nhất.

1.5.15 Mỗi quan hệ: File — parses_into — CodeEntity

- **Phân loại thực thể:** File (Thực thể yếu) ↔ CodeEntity (Thực thể yếu – phụ thuộc File)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** File (0..*) ↔ CodeEntity (1..1)
- **Mức tham gia:** File: Tùy chọn (Partial) | CodeEntity: Bắt buộc (Total)

- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một File code qua bóc tách AST thu được nhiều thực thể code (Class, Function); mỗi thực thể code nằm trọn trong đúng một File.

1.5.16 Mối quan hệ: CodeEntity — depends_on — CodeEntity

- **Phân loại thực thể:** CodeEntity (Thực thể yếu) ↔ CodeEntity (Thực thể yếu)
- **Tỷ lệ bản số:** M - N
- **Tên mối kết hợp đệ quy:** <depends_on>
- **Bản số chi tiết (Theo vai trò):**
 - Vai trò Phụ thuộc (Dependent): 0..*
 - Vai trò Được phụ thuộc (Target): 0..*
- **Mức tham gia:** Cả 2 phía đều là Tùy chọn (Partial)
- **Thuộc tính mối quan hệ:** dependency_type (ví dụ: import, function_call, inherits).
- **Mô tả nghiệp vụ:** Xác định mối quan hệ phụ thuộc (gọi hàm, kế thừa, import) giữa các đơn vị code trong hệ thống.

1.5.17 Nhóm Khuôn mẫu Tài liệu (Template Engine Domain)

1.5.18 Mối quan hệ: Workspace — defines — Template

- **Phân loại thực thể:** Workspace (Thực thể mạnh) ↔ Template (Thực thể mạnh)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** Workspace (0..*) ↔ Template (1..1)
- **Mức tham gia:** Workspace: Tùy chọn (Partial) | Template: Bắt buộc (Total)
- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một Workspace định nghĩa nhiều khuôn mẫu tài liệu; mỗi mẫu tài liệu thuộc sở hữu của một Workspace.

1.5.19 Mỗi quan hệ: **Template** — **has_section** — **TemplateSection**

- **Phân loại thực thể:** Template (Thực thể mạnh) ↔ TemplateSection (Thực thể yếu – phụ thuộc Template)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** Template (1..*) ↔ TemplateSection (1..1)
- **Mức tham gia:** Template: Bắt buộc (Total) | TemplateSection: Bắt buộc (Total)
- **Thuộc tính mỗi quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một Template cấu thành từ ít nhất một Section dần ý; mỗi Section nằm trên đúng một Template.

1.5.20 Mỗi quan hệ: **TemplateSection** — **contains_placeholder** — **Placeholder**

- **Phân loại thực thể:** TemplateSection (Thực thể yếu – phụ thuộc Template) ↔ Placeholder (Thực thể yếu – phụ thuộc TemplateSection)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** TemplateSection (0..*) ↔ Placeholder (1..1)
- **Mức tham gia:** TemplateSection: Tùy chọn (Partial) | Placeholder: Bắt buộc (Total)
- **Thuộc tính mỗi quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Mỗi mục trong mẫu tài liệu chứa nhiều biến chèn tự động. Khi xóa một Section, tất cả các Placeholder nằm trong Section đó cũng bị xóa theo.

1.5.21 Mỗi quan hệ: **Placeholder** — **maps_to** — **CodeEntity**

- **Phân loại thực thể:** Placeholder (Thực thể yếu) ↔ CodeEntity (Thực thể yếu)
- **Tên mỗi kết hợp:** <maps_to>
- **Tỷ lệ bản số:** M - N

- **Bản số chi tiết:** Placeholder (0..*) $\leftrightarrow$ TemplateMapping (1..*) | CodeEntity (0..*) $\leftrightarrow$ TemplateMapping (1..1)
- **Mức tham gia:** Placeholder: Tùy chọn (Partial) | CodeEntity: Tùy chọn (Partial)
- **Thuộc tính mối quan hệ:** extraction_rule, is_active.
- **Mô tả nghiệp vụ:** Ánh xạ chỉ thị cho AI biết lấy thông tin từ thành phần code nào để điền vào biến trong mẫu.

1.5.22 Nhóm Tài liệu & Vòng đời Dữ liệu (Documentation Domain)

1.5.23 Mối quan hệ: Workspace — manages — Document

- **Phân loại thực thể:** Workspace (Thực thể mạnh) $\leftrightarrow$ Document (Thực thể mạnh)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** Workspace (0..*) $\leftrightarrow$ Document (1..1)
- **Mức tham gia:** Workspace: Tùy chọn (Partial) | Document: Bắt buộc (Total)
- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Workspace quản lý danh mục tài liệu gốc của dự án; một tài liệu thuộc về đúng một Workspace.

1.5.24 Mối quan hệ: User — authors — Document

- **Phân loại thực thể:** User (Thực thể mạnh) $\leftrightarrow$ Document (Thực thể mạnh)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** User (0..*) $\leftrightarrow$ Document (1..1)
- **Mức tham gia:** User: Tùy chọn (Partial) | Document: Bắt buộc (Total)
- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Người dùng đứng tên tác giả khởi tạo tài liệu; mỗi tài liệu có một tác giả duy nhất.

1.5.25 Mỗi quan hệ: Document — has_version — DocumentVersion

- **Phân loại thực thể:** Document (Thực thể mạnh) ↔ DocumentVersion (Thực thể yếu – phụ thuộc Document)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** Document (1..*) ↔ DocumentVersion (1..1)
- **Mức tham gia:** Document: Bắt buộc (Total) | DocumentVersion: Bắt buộc (Total)
- **Thuộc tính mỗi quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một tài liệu chính thức chứa ít nhất một phiên bản nội dung; mỗi phiên bản đại diện cho một mốc chỉnh sửa của tài liệu gốc.

1.5.26 Mỗi quan hệ: DocumentVersion — tracks_change — ChangeLog

- **Phân loại thực thể:** DocumentVersion (Thực thể yếu) ↔ ChangeLog (Thực thể yếu – phụ thuộc DocumentVersion)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** DocumentVersion (0..*) ↔ ChangeLog (1..1)
- **Mức tham gia:** DocumentVersion: Tùy chọn (Partial) | ChangeLog: Bắt buộc (Total)
- **Thuộc tính mỗi quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Mỗi phiên bản tài liệu có thể ghi nhận nhiều lượt nhật ký biến động nội dung chi tiết.

1.5.27 Nhóm Kiểm duyệt & Phê duyệt (Review & Governance Domain)

1.5.28 Mỗi quan hệ: DocumentVersion — has_review — Review

- **Phân loại thực thể:** DocumentVersion (Thực thể yếu) ↔ Review (Thực thể yếu – phụ thuộc DocumentVersion)

- **Tỷ lệ bản số:**
- **Bản số chi tiết:** DocumentVersion (0..*) $\leftrightarrow$ Review (1..1)
- **Mức tham gia:** DocumentVersion: Tùy chọn (Partial) | Review: Bắt buộc (Total)
- **Thuộc tính thực thể Review:** status, decision, summary_notes, created_at.
- **Mô tả nghiệp vụ:** Một phiên bản tài liệu có thể nhận 0 hoặc nhiều lượt đánh giá kỹ thuật; mỗi lượt đánh giá bắt buộc phải thuộc về duy nhất một phiên bản tài liệu.

1.5.29 Mối quan hệ: User — performs_review — Review

- **Phân loại thực thể:** User (Thực thể mạnh) $\leftrightarrow$ Review (Thực thể yếu)
- **Tỷ lệ bản số:**
- **Bản số chi tiết:** User (0..*) $\leftrightarrow$ Review (1..1)
- **Mức tham gia:** User: Tùy chọn (Partial) | Review: Bắt buộc (Total)
- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một người dùng có thể thực hiện 0 hoặc nhiều lượt đánh giá; mỗi lượt review bắt buộc do đúng một người dùng chịu trách nhiệm.

1.5.30 Mối quan hệ: Review — contains_comment — ReviewComment

- **Phân loại thực thể:** Review (Thực thể sự kiện) $\leftrightarrow$ ReviewComment (Thực thể yếu – phụ thuộc Review)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** Review (0..*) $\leftrightarrow$ ReviewComment (1..1)
- **Mức tham gia:** Review: Tùy chọn (Partial) | ReviewComment: Bắt buộc (Total)
- **Thuộc tính mối quan hệ:** Không có.

- **Mô tả nghiệp vụ:** Một phiên kiểm duyệt kỹ thuật chứa nhiều bình luận góp ý chi tiết; mỗi bình luận gắn chặt với một lượt review.

1.5.31 Mỗi quan hệ: **DocumentVersion** — **requires_approval** — **Approval**

- **Phân loại thực thể:** DocumentVersion (Thực thể yếu) ↔ Approval (Thực thể yếu – phụ thuộc DocumentVersion)
- **Tỷ lệ bản số:**
- **Bản số chi tiết:** DocumentVersion (0..*) ↔ Approval (1..1)
- **Mức tham gia:** DocumentVersion: Tùy chọn (Partial) | Approval: Bắt buộc (Total)
- **Thuộc tính thực thể Approval:** decision, rejection_reason, approved_at.
- **Mô tả nghiệp vụ:** Một phiên bản tài liệu có thể có 0 hoặc nhiều lượt phê duyệt; mỗi lượt phê duyệt bắt buộc thuộc về đúng một phiên bản tài liệu cụ thể.

1.5.32 Mỗi quan hệ: **User** — **grants_approval** — **Approval**

- **Phân loại thực thể:** User (Thực thể mạnh) ↔ Approval (Thực thể yếu)
- **Tỷ lệ bản số:**
- **Bản số chi tiết:** User (0..*) ↔ Approval (1..1)
- **Mức tham gia:** User: Tùy chọn (Partial) | Approval: Bắt buộc (Total)
- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Một người dùng (Manager) có thể thực hiện 0 hoặc nhiều đợt phê duyệt; mỗi bản ghi phê duyệt bắt buộc do đúng một người dùng chịu trách nhiệm ra quyết định.

1.5.33 Nhóm AI Evidence, Giám sát & Kiểm toán (AI Evidence & Observability Domain)

1.5.34 Mối quan hệ: DocumentVersion — has_evidence — GroundingEvidence

- **Phân loại thực thể:** DocumentVersion (Thực thể yếu) ↔ GroundingEvidence (Thực thể yếu – phụ thuộc DocumentVersion)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** DocumentVersion (0..*) ↔ GroundingEvidence (1..1)
- **Mức tham gia:** DocumentVersion: Tùy chọn (Partial) | GroundingEvidence: Bắt buộc (Total)
- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Phiên bản tài liệu lưu trữ bằng minh chứng đối soát nguồn gốc code để chống ảo giác (hallucination) của AI.

1.5.35 Mối quan hệ: GroundingEvidence — references_code — CodeEntity

- **Phân loại thực thể:** GroundingEvidence (Thực thể yếu) ↔ CodeEntity (Thực thể yếu)
- **Tỷ lệ bản số:** M - N
- **Tên mối kết hợp:** <references_code>
- **Bản số chi tiết:** GroundingEvidence (0..*) ↔ CodeEntity (0..*)
- **Mức tham gia:** GroundingEvidence: Tùy chọn (Partial) | CodeEntity: Tùy chọn (Partial)
- **Thuộc tính mối quan hệ:** similarity_score, match_type.
- **Mô tả nghiệp vụ:** Minh chứng AI liên kết trực tiếp với các đơn vị code cụ thể trong hệ thống để phục vụ đối soát sai lệch.

1.5.36 **Mối quan hệ: GroundingEvidence — detects — DriftAlert**

- **Phân loại thực thể:** GroundingEvidence (Thực thể yếu) ↔ DriftAlert (Thực thể sự kiện)
- **Tỷ lệ bản số:** 1 - N
- **Bản số chi tiết:** GroundingEvidence (0..*) ↔ DriftAlert (1..1)
- **Mức tham gia:** GroundingEvidence: Tùy chọn (Partial) | DriftAlert: Bắt buộc (Total)
- **Thuộc tính mối quan hệ:** Không có.
- **Mô tả nghiệp vụ:** Bằng chứng đối soát khi phát hiện sự bất đồng giữa tài liệu và mã nguồn mới sẽ kích hoạt cảnh báo sai lệch (Drift Alert).

Chương 2

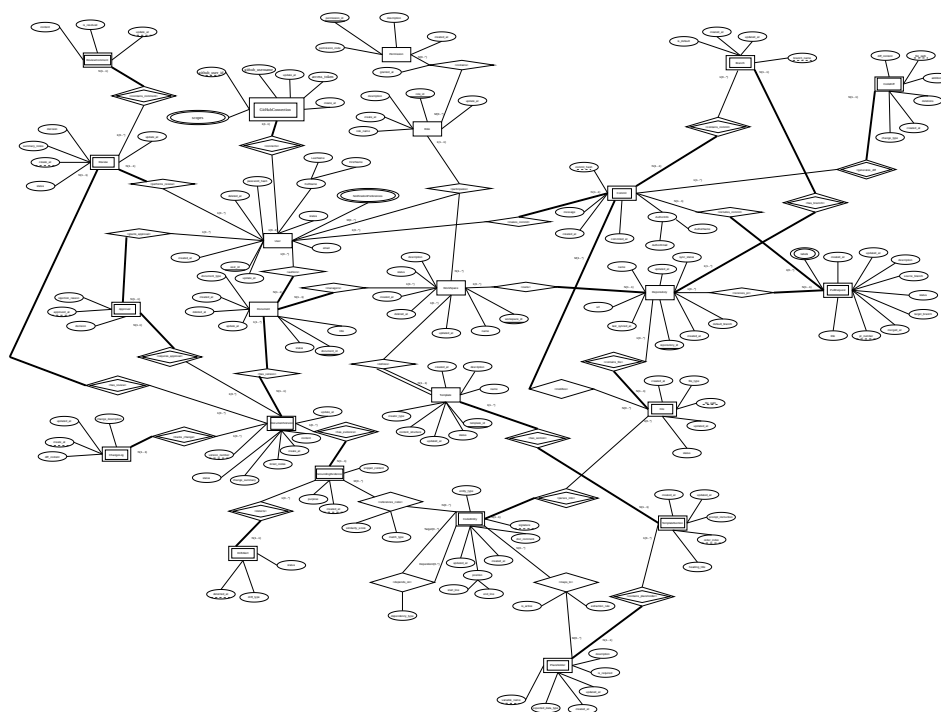
Sơ đồ Quan niệm ERD / UML Class Diagram

Đầu ra của Stage 1 (Thiết kế quan niệm) bao gồm sơ đồ ERD và UML Class Diagram, biểu diễn trừu tượng toàn bộ các thực thể, thuộc tính và mối kết hợp nghiệp vụ của hệ thống LivingDocs ở mức logic cao nhất, hoàn toàn độc lập với bất kỳ hệ quản trị CSDL vật lý nào.

Mô hình đã xác định chính xác các bản số ($1 : 1$, $1 : N$, $M : N$) và quy tắc ràng buộc miền, bao phủ trọn vẹn từ quản trị không gian làm việc, cấu trúc kho code cho đến luồng kiểm duyệt 2 cấp ($\text{Staff} \rightarrow \text{Manager}$) và cơ chế lưu minh chứng AI chống ảo giác.

Đây là bức tranh kiến trúc dữ liệu hoàn chỉnh, đóng vai trò là đầu vào (input) chuẩn xác để thực hiện ánh xạ sang Lược đồ quan hệ (Logical Schema) và chuẩn hóa 3NF ở Stage 2.

2.1 Sơ đồ ERD mức khái niệm



Hình 2.1: Sơ đồ ERD mức khái niệm

Phần II

Thiết kế Logic

Chương 3

Áp dụng Thuật toán 2.2 (Lược đồ Quan hệ & Chuẩn hóa) 3NF

Ở mức thiết kế logic (*Logical Design Level*), mục tiêu cốt lõi là chuyển hóa mô hình dữ liệu quan niệm trừu tượng (ERD/UML) thành lược đồ quan hệ lô-gíc (*Logical Relational Schema*) cụ thể, xác định rõ cấu trúc các bảng, cột, khóa chính (PK), khóa ngoại (FK) và các ràng buộc toàn vẹn [1, 2]. Đồ án áp dụng **Thuật toán 2.2** (*Algorithm 2.2: Ánh xạ ERD sang Lược đồ Quan hệ*) để đảm bảo tính toàn vẹn tham chiếu, loại bỏ các dị thường dữ liệu và đưa toàn bộ lược đồ đạt tiêu chuẩn chuẩn hóa 3NF cho hệ thống **LivingDocs — Design and Implementation of an AI-Assisted Self-Updating Document System for Engineering Teams** [3-5].

Quá trình này tiếp nhận đầu vào (*Input*) là Sơ đồ Quan niệm (*Conceptual ERD/UML*) đã qua kiểm chứng ở Stage 1, và kết xuất đầu ra (*Output*) là Lược đồ Quan hệ Lô-gíc (*Logical Schema*) hoàn chỉnh, hồ sơ Ràng buộc Khóa ngoại/Giá trị miền (*FK & CHECK Rules*) và Báo cáo chứng minh Chuẩn hóa 3NF [3, 5-12].

Quy trình Thuật toán 2.2 (*Relational Mapping Algorithm*):

- **Đầu vào (*Input*):** Sơ đồ Quan niệm ERD/UML và từ điển dữ liệu hoàn chỉnh từ Stage 1 .
- **Bước 1 (Thực thể mạnh):** Ánh xạ các thực thể mạnh thành các bảng chính, chuyển thuộc tính đơn thành cột và thiết lập Khóa chính (PK) .

- **Bước 2 (Thực thể yếu):** Ánh xạ các thực thể yếu thành các bảng phụ thuộc, gán Surrogate PK kiểu UUID và cài đặt FK trở về thực thể chủ .
- **Bước 3 & 4 (Mối kết hợp 1 : 1 và 1 : N):** Chèn Khóa ngoại (FK) từ bảng phía 1 sang bảng phía N (hoặc phía tham gia bắt buộc) kèm các ràng buộc NOT NULL/UNIQUE phù hợp .
- **Bước 5 (Mối kết hợp $M : N$):** Khởi tạo các bảng trung gian (*Junction Tables*) chứa cặp Khóa ngoại (FK) trở về hai bảng gốc và các thuộc tính riêng của mối kết hợp .
- **Bước 6 (Thuộc tính đặc biệt):** Phẳng hóa các thuộc tính phức hợp thành các cột đơn độc lập và tách các thuộc tính đa trị thành các bảng phụ riêng .
- **Bước 7 (Ràng buộc & Chuẩn hóa):** Khai báo tập Phụ thuộc hàm (FDs), chứng minh lược đồ đạt chuẩn 3NF và thiết lập các ràng buộc toàn vẹn (*FK Constraints*) .
- **Đầu ra (*Output*):** Lược đồ Quan hệ Lô-gíc (*Logical Schema*) hoàn chỉnh đạt chuẩn 3NF và hồ sơ ràng buộc toàn vẹn .

3.1 Ánh xạ Thực thể, Mối kết hợp & Phẳng hóa Thuộc tính (Thuật toán 2.2)

Bước 1: Thực thể mạnh $\rightarrow$ Bảng chính

Chuyển 7 thực thể mạnh thành 7 bảng chính, đặt tên theo chuẩn `snake_case` số nhiều:

- `users`, `workspaces`, `roles`, `permissions`, `repositories`, `templates`, `documents`.

Bước 2: Thực thể yếu $\rightarrow$ Bảng phụ & Mối kết hợp $M : N \rightarrow$ Bảng trung gian (Surrogate PK UUID)

Chuyển 16 thực thể yếu thành 16 bảng phụ, mỗi bảng được gán khóa chính nhân tạo (Surrogate PK) kiểu UUID độc lập với bảng cha, mang FK định danh (identifying FK) trở về đúng 1 bảng cha:

- `github_connections` (phụ thuộc `users`)

3.1. ÁNH XẠ THỰC THỂ, MỐI KẾT HỢP & PHẪNG HÓA THUỘC TÍNH (THUẬT TOÁN 2.2) 49

- `branches` (phụ thuộc `repositories`)
- `commits` (phụ thuộc `branches`; FK phụ `author_user_id` → `users`, `pr_id` → `pull_requests`)
- `pull_requests` (phụ thuộc `repositories`)
- `code_diffs` (phụ thuộc `commits`)
- `files` (phụ thuộc `repositories`)
- `code_entities` (phụ thuộc `files`)
- `template_sections` (phụ thuộc `templates`)
- `placeholders` (phụ thuộc `template_sections`)
- `document_versions` (phụ thuộc `documents`)
- `change_logs` (phụ thuộc `document_versions`)
- `reviews` (phụ thuộc `document_versions`; FK phụ `reviewer_user_id` → `users`)
- `review_comments` (phụ thuộc `reviews`)
- `approvals` (phụ thuộc `document_versions`; FK phụ `approver_user_id` → `users`)
- `grounding_evidences` (phụ thuộc `document_versions`)
- `drift_alerts` (phụ thuộc `grounding_evidences`)

Song song đó, chuyển các mối kết hợp $M : N$ (bao gồm 1 mối kết hợp 3 ngôi và 1 mối kết hợp đệ quy) thành 6 bảng trung gian, mỗi bảng được gán Surrogate PK riêng kiểu `UUID`, chứa tối thiểu 2 FK cùng vai trò định danh trở về các bảng gốc:

- `workspace_members` (mối kết hợp 3 ngôi *participates*: `user_id`, `workspace_id`, `role_id`)
- `role_permissions` (mối kết hợp *contains*: `role_id`, `permission_id`)
- `commit_file` (mối kết hợp *modifies*: `commit_id`, `file_id`)
- `dependencies` (mối kết hợp đệ quy *depends_on*: `dependent_entity_id`, `target_entity_id` — cả hai cùng trở về `code_entities`)

- `template_mappings` (mỗi kết hợp `maps_to:placeholder_id, code_entity_id`)
- `grounding_evidence_code_entities` (mỗi kết hợp `references_code:evidence_id, code_entity_id`)

Bước 3 & 4: Quan hệ 1:1, 1:N → Chèn FK

Nguyên tắc: Chuyển khóa chính (PK) của bảng phía 1 sang làm khóa ngoại (FK) ở bảng phía N. Với quan hệ 1:1, đặt khóa ngoại ở phía tham gia tùy chọn và gán ràng buộc **UNIQUE** để đảm bảo tính một-một.

- `users.user_id` (PK) → `github_connections.user_id` (FK) (1:1 nghiệp vụ, ràng buộc **UNIQUE**)
- `workspaces.workspace_id` (PK) → `repositories.workspace_id` (FK) (1:N)
- `repositories.repository_id` (PK) → `branches.repository_id` (FK) (1:N)
- `branches.branch_id` (PK) → `commits.branch_id` (FK) (1:N)
- `commits.author_user_id` (FK) → `users.user_id` (PK) (1:N)
- `commits.pr_id` (FK) → `pull_requests.pr_id` (PK) (1:N, cho phép NULL)
- `pull_requests.repository_id` (FK) → `repositories.repository_id` (PK) (1:N)
- `code_diffs.commit_id` (FK) → `commits.commit_id` (PK) (1:N)
- `files.repository_id` (FK) → `repositories.repository_id` (PK) (1:N)
- `code_entities.file_id` (FK) → `files.file_id` (PK) (1:N)
- `templates.workspace_id` (FK) → `workspaces.workspace_id` (PK) (1:N)
- `template_sections.template_id` (FK) → `templates.template_id` (PK) (1:N)
- `placeholders.section_id` (FK) → `template_sections.section_id` (PK) (1:N)

3.1. ÁNH XẠ THỰC THỂ, MỐI KẾT HỢP & PHẪNG HÓA THUỘC TÍNH (THUẬT TOÁN 2.2) 51

- `documents.workspace_id` (FK) → `workspaces.workspace_id` (PK) (1:N)
- `documents.author_user_id` (FK) → `users.user_id` (PK) (1:N)
- `document_versions.document_id` (FK) → `documents.document_id` (PK) (1:N)
- `change_logs.version_id` (FK) → `document_versions.version_id` (PK) (1:N)
- `reviews.version_id` (FK) → `document_versions.version_id` (PK) (1:N)
- `reviews.reviewer_user_id` (FK) → `users.user_id` (PK) (1:N)
- `review_comments.review_id` (FK) → `reviews.review_id` (PK) (1:N)
- `approvals.version_id` (FK) → `document_versions.version_id` (PK) (1:N)
- `approvals.approver_user_id` (FK) → `users.user_id` (PK) (1:N)
- `grounding_evidences.version_id` (FK) → `document_versions.version_id` (PK) (1:N)
- `drift_alerts.evidence_id` (FK) → `grounding_evidences.evidence_id` (PK) (1:N)

Bước 5: Quan hệ M:N → Bảng trung gian

Tạo 6 bảng trung gian, mỗi bảng có Surrogate PK riêng kiểu UUID và tối thiểu 2 FK trở về các bảng gốc:

- `workspace_members`: chứa FK `user_id`, `workspace_id`, `role_id` (mỗi kết hợp 3 ngôi *participates*, tỷ lệ M:N:1)
- `role_permissions`: chứa FK `role_id`, `permission_id` (mỗi kết hợp *contains*, M:N)
- `commit_file`: chứa FK `commit_id`, `file_id` (mỗi kết hợp *modifies*, M:N)
- `dependencies`: chứa FK `dependent_entity_id`, `target_entity_id` (mỗi kết hợp đệ quy *depends_on*, M:N)

- `template_mappings`: chứa FK `placeholder_id`, `code_entity_id` (mỗi kết hợp `maps_to`, M:N)
- `grounding_evidence_code_entities`: chứa FK `evidence_id`, `code_entity_id` (mỗi kết hợp `references_code`, M:N)

Bước 6: Thuộc tính đặc biệt

- **Phẳng hóa thuộc tính phức hợp:**
 - `User.FullName` → `users.first_name`, `users.last_name`
 - `Commit.AuthorInfo` → `commits.author_name`, `commits.author_email`
 - `CodeEntity.position` → `code_entities.start_line`, `code_entities.end_line`
- **Tách thuộc tính đa trị thành bảng phụ chứa FK trở về bảng chính:**
 - `User.NotificationPreferences` → `user_notification_preferences`
 - `GitHubConnection.scopes` → `github_connection_scopes`
 - `PullRequest.labels` → `pull_request_labels`

;

3.2 Lược đồ Quan hệ Lô-gíc (Logical Schema) & 3NF-Phân hệ Quản trị & Xác thực

1. Bảng `users` (Bảng chính - Thực thể mạnh: User)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
<code>user_id</code>	UUID	PK	NOT NULL
<code>email</code>	-	Unique	NOT NULL
<code>password_hash</code>	-	-	NULL
<code>first_name</code>	-	-	NULL
<code>last_name</code>	-	-	NULL
<code>status</code>	-	-	NOT NULL
<code>created_at</code>	-	-	NOT NULL
<code>updated_at</code>	-	-	NOT NULL
<code>deleted_at</code>	-	-	NULL

3.2. LƯỢC ĐỒ QUAN HỆ LÔ-GÍC (LOGICAL SCHEMA) & 3NF-PHÂN HỆ QUẢN TRỊ & XÁC THỰC

2. Bảng workspaces (Bảng chính - Thực thể mạnh: Workspace)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
workspace_id	UUID	PK	NOT NULL
name	-	-	NOT NULL
description	-	-	NULL
status	-	-	NOT NULL
created_at	-	-	NOT NULL
updated_at	-	-	NOT NULL
deleted_at	-	-	NULL

3. Bảng roles (Bảng chính - Thực thể mạnh: Role)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
role_id	UUID	PK	NOT NULL
role_name	-	-	NOT NULL
description	-	-	NULL
created_at	-	-	NOT NULL
updated_at	-	-	NOT NULL

4. Bảng permissions (Bảng chính - Thực thể mạnh: Permission)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
permission_id	UUID	PK	NOT NULL
permission_code	-	Unique	NOT NULL
description	-	-	NULL
created_at	-	-	NOT NULL

5. Bảng github_connections (Bảng phụ - Thực thể yếu phụ thuộc User, Surrogate PK UUID)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
connection_id	UUID	PK	NOT NULL
user_id	-	FK, Unique	NOT NULL
github_user_id	-	-	NOT NULL
github_username	-	-	NOT NULL
access_token	-	-	NOT NULL
created_at	-	-	NOT NULL
updated_at	-	-	NOT NULL
Ràng buộc bổ sung: UNIQUE (user_id)			

6. Bảng user_notification_preferences (Bảng phụ đa trị từ NotificationPreferences của User)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
preference_id	UUID	PK	NOT NULL
user_id	-	FK	NOT NULL
channel	-	-	NOT NULL
Ràng buộc bổ sung: UNIQUE (user_id, channel) <i>Một người dùng chỉ có tối đa 1 bản ghi cho mỗi kênh.</i>			

7. Bảng github_connection_scopes (Bảng phụ đa trị từ scopes của GitHubConnection)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
scope_id	UUID	PK	NOT NULL
connection_id	-	FK	NOT NULL
scope_value	-	-	NOT NULL
Ràng buộc bổ sung: UNIQUE (connection_id, scope_value) <i>Đảm bảo tài khoản OAuth không lưu lặp lại cùng một quyền scope.</i>			

8. Bảng workspace_members (Bảng trung gian mối kết hợp 3 ngôi User–Workspace–Role, tỷ lệ M:N:1)

3.2. LƯỢC ĐỒ QUAN HỆ LÔ-GÍC (LOGICAL SCHEMA) & 3NF-PHÂN HỆ QUẢN TRỊ & XÁC THỰC55

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
membership_id	UUID	PK	NOT NULL
user_id	-	FK	NOT NULL
workspace_id	-	FK	NOT NULL
role_id	-	FK	NOT NULL
joined_at	-	-	NOT NULL
status	-	-	NOT NULL
Ràng buộc bổ sung: UNIQUE (user_id, workspace_id) Đảm bảo mỗi User chỉ giữ đúng 1 Role trong 1 Workspace (Scoped RBAC).			

9. Bảng role_permissions (Bảng trung gian mỗi kết hợp Role-Permission, tỷ lệ M:N)

Tên Cột	Kiểu Dữ Liệu	Khóa	Ràng Buộc
role_permission_id	UUID	PK	NOT NULL
role_id	-	FK	NOT NULL
permission_id	-	FK	NOT NULL
granted_at	-	-	NOT NULL
Ràng buộc bổ sung: UNIQUE (role_id, permission_id) Ngăn chặn gán trùng lặp quyền cho một vai trò (RBAC).			

3.2.1 Lược đồ Cú pháp Rút gọn & Chứng minh Chuẩn hóa 3NF

Lược đồ Dạng Cú pháp Rút gọn

- users (user_id [UUID, PK], email [Unique], password_hash, first_name, last_name, status, created_at, updated_at, deleted_at)
- workspaces (workspace_id [UUID, PK], name, description, status, created_at, updated_at, deleted_at)
- roles (role_id [UUID, PK], role_name, description, created_at, updated_at)
- permissions (permission_id [UUID, PK], permission_code [Unique], description, created_at)
- github_connections (connection_id [UUID, PK], user_id [FK, Unique], github_user_id, github_username, access_token, created_at, updated_at)

- **user_notification_preferences** (preference_id [UUID, PK], user_id [FK], channel, UNIQUE(user_id, channel))
- **github_connection_scopes** (scope_id [UUID, PK], connection_id [FK], scope_value, UNIQUE(connection_id, scope_value))
- **workspace_members** (membership_id [UUID, PK], user_id [FK], workspace_id [FK], role_id [FK], joined_at, status, UNIQUE(user_id, workspace_id))
- **role_permissions** (role_permission_id [UUID, PK], role_id [FK], permission_id [FK], granted_at, UNIQUE(role_id, permission_id))

3.2.2 Báo cáo Chứng minh Chuẩn hóa 3NF

Khai báo Tập Phụ thuộc Hàm (Functional Dependencies - FDs)

Xét các bảng trong phân hệ Quản trị, Người dùng & Phân quyền có tập phụ thuộc hàm được xác định chi tiết như sau:

1. Bảng users

- $f_1 : \text{user_id} \rightarrow \text{email, password_hash, first_name, last_name, status, created_at, updated_at}$
- $f_2 : \text{email} \rightarrow \text{user_id, password_hash, first_name, last_name, status, created_at, updated_at}$
(Khóa ứng viên)

2. Bảng user_notification_preferences

- $f_1 : \text{preference_id} \rightarrow \text{user_id, channel}$
- $f_2 : (\text{user_id, channel}) \rightarrow \text{preference_id}$

3. Bảng workspaces

- $f_1 : \text{workspace_id} \rightarrow \text{name, description, status, created_at, updated_at, deleted_at}$

4. Bảng roles

- $f_1 : \text{role_id} \rightarrow \text{role_name, description, created_at, updated_at}$

5. Bảng permissions

- $f_1 : \text{permission_id} \rightarrow \text{permission_code, description, created_at}$
- $f_2 : \text{permission_code} \rightarrow \text{permission_id, description, created_at}$ (Khóa ứng viên)

3.2. LƯỢC ĐỒ QUAN HỆ LÔ-GÍC (LOGICAL SCHEMA) & 3NF-PHÂN HỆ QUẢN TRỊ & XÁC THỰC

6. Bảng `role_permissions`

- $f_1 : \text{role_permission_id} \rightarrow \text{role_id}, \text{permission_id}, \text{granted_at}$
- $f_2 : (\text{role_id}, \text{permission_id}) \rightarrow \text{role_permission_id}, \text{granted_at}$

7. Bảng `workspace_members`

- $f_1 : \text{membership_id} \rightarrow \text{user_id}, \text{workspace_id}, \text{role_id}, \text{joined_at}, \text{status}$
- $f_2 : (\text{user_id}, \text{workspace_id}) \rightarrow \text{membership_id}, \text{role_id}, \text{joined_at}, \text{status}$

8. Bảng `github_connections`

- $f_1 : \text{connection_id} \rightarrow \text{user_id}, \text{github_user_id}, \text{github_username}, \text{access_token}, \text{created_at}, \text{update_at}$
- $f_2 : \text{user_id} \rightarrow \text{connection_id}, \text{github_user_id}, \text{github_username}, \text{access_token}, \text{created_at}, \text{update_at}$
(Khóa ứng viên)

9. Bảng `github_connection_scopes`

- $f_1 : \text{scope_id} \rightarrow \text{connection_id}, \text{scope_value}$
- $f_2 : (\text{connection_id}, \text{scope_value}) \rightarrow \text{scope_id}$

Đánh giá Cấp Chuẩn (Normal Forms Assessment)

• Đạt Chuẩn 1NF (First Normal Form):

- *Giá trị nguyên tố (Atomic Values):* Toàn bộ các cột trong tất cả các bảng đều lưu giữ giá trị đơn, không thể phân chia nhỏ hơn.
- *Phẳng hóa thuộc tính phức hợp:* Thuộc tính phức hợp `FullName` đã được tách thành hai thuộc tính nguyên tố là `first_name` và `last_name`.
- *Tách thuộc tính đa trị:* Các thuộc tính đa trị như danh sách cài đặt thông báo (`NotificationPreferences`) và danh sách quyền truy cập (`scopes`) đã được tách thành các bảng phụ tương ứng là `user_notification_preferences` và `github_connection_scopes`.

• Đạt Chuẩn 2NF (Second Normal Form):

- *Sử dụng Khóa chính đơn (Surrogate PK):* Mọi bảng trong phân hệ đều thiết lập Khóa chính đơn định danh bằng chuỗi UUID (`user_id, workspace_id, role_id, membership_id, ...`).

- *Loại bỏ phụ thuộc một phần*: Do không dùng Khóa chính phức hợp làm khóa chính duy nhất, mọi thuộc tính không khóa đều phụ thuộc đầy đủ vào toàn bộ Khóa chính, loại bỏ hoàn toàn hiện tượng phụ thuộc một phần (Partial Dependency).

- **Đạt Chuẩn 3NF (Third Normal Form):**

- *Phụ thuộc trực tiếp*: Tất cả thuộc tính không khóa đều phụ thuộc trực tiếp vào Khóa chính của bảng chứa nó.
- *Loại bỏ phụ thuộc bắc cầu*: Không tồn tại phụ thuộc bắc cầu dạng $X \rightarrow Y \rightarrow Z$ giữa các thuộc tính không khóa. Ví dụ: Thông tin phân quyền giữa nhóm quyền và người dùng không lưu trực tiếp trong `users` hay `roles` mà được tách riêng thành các bảng trung gian `role_permissions` và `workspace_members`.

3.3 Lược đồ Quan hệ Lô-gíc (Logical Schema) & 3NF-Phân hệ Mã nguồn & Cấu trúc Code

Khai báo Chi tiết Bảng

Tên Bảng: repositories

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
repository_id	UUID	PK	NOT NULL
workspace_id		FK	NOT NULL
name			NOT NULL
url		Unique	NOT NULL
default_branch			NOT NULL
sync_status			NOT NULL
last_synced_at			NULL
created_at			NOT NULL
updated_at			NOT NULL

Tên Bảng: branches

3.3. LƯỢC ĐỒ QUAN HỆ LÔ-GÍC (LOGICAL SCHEMA) & 3NF-PHÂN HỆ MÃ NGUỒN & CẤU TRÚC C

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
branch_id	UUID	PK	NOT NULL
repository_id		FK	NOT NULL
branch_name			NOT NULL
is_default			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Tên Bảng: commits

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
commit_id	UUID	PK	NOT NULL
branch_id		FK	NOT NULL
user_id		FK	NULL
commit_hash		Unique	NOT NULL
message			NOT NULL
author_name			NOT NULL
author_email			NOT NULL
committed_at			NOT NULL
created_at			NOT NULL

Tên Bảng: pull_requests

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
pr_id	UUID	PK	NOT NULL
repository_id		FK	NOT NULL
pr_number			NOT NULL
title			NOT NULL
description			NOT NULL
source_branch			NOT NULL
target_branch			NOT NULL
status			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL
merged_at			NULL

Tên Bảng: pull_request_labels

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
label_id	UUID	PK	NOT NULL
pr_id		FK	NOT NULL
label_name			NOT NULL

Tên Bảng: code_diffs

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
diff_id	UUID	PK	NOT NULL
commit_id		FK	NULL
pr_id		FK	NOT NULL
file_path			NOT NULL
change_type			NOT NULL
additions			NOT NULL
deletions			NOT NULL
diff_content			NULL
created_at			NOT NULL

Tên Bảng: files

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
file_id	UUID	PK	NOT NULL
repository_id		FK	NOT NULL
file_path			NOT NULL
file_type			NULL
status			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Tên Bảng: code_entities

3.3. LƯỢC ĐỒ QUAN HỆ LÔ-GÍC (LOGICAL SCHEMA) & 3NF-PHÂN HỆ MÃ NGUỒN & CẤU TRÚC C

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
entity_id	UUID	PK	NOT NULL
file_id		FK	NOT NULL
entity_name			NOT NULL
entity_type			NOT NULL
signature			NULL
doc_comment			NULL
start_line			NOT NULL
end_line			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Tên Bảng: dependencies

Tên Cột	Kiểu Dữ liệu	Khóa	Ràng Buộc
dependency_id	UUID	PK	NOT NULL
source_entity_id		FK	NOT NULL
target_entity_id		FK	NOT NULL
dependency_type			NOT NULL
created_at			NOT NULL

3.3.1 Lược đồ Dạng Cú pháp Rút gọn

- **repositories** (repository_id UUID PK NOT NULL, workspace_id UUID FK NOT NULL, name NOT NULL, url NOT NULL, default_branch NOT NULL, sync_status NOT NULL, last_synced_at TIMESTAMP NULL, created_at NOT NULL, updated_at NOT NULL, UNIQUE(url))
- **branches** (branch_id UUID PK NOT NULL, repository_id UUID FK NOT NULL, branch_name NOT NULL, is_default NOT NULL, created_at NOT NULL, updated_at NOT NULL, UNIQUE(repository_id, branch_name))
- **commits** (commit_id UUID PK NOT NULL, branch_id UUID FK NOT NULL, user_id UUID FK NULL, commit_hash NOT NULL, message TEXT NOT NULL, author_name NOT NULL, author_email NOT NULL, committed_at NOT NULL, created_at NOT NULL, UNIQUE(commit_hash))
- **pull_requests** (pr_id UUID PK NOT NULL, repository_id UUID

FK NOT NULL, pr_number NOT NULL, title NOT NULL, description NOT NULL, source_branch NOT NULL, target_branch VARCHAR(100) NOT NULL, status VARCHAR(50) NOT NULL, created_at NOT NULL, updated_at NOT NULL, merged_at NULL, UNIQUE(repository_id, pr_number))

- **pull_request_labels** (label_id UUID PK NOT NULL, pr_id UUID FK NOT NULL, label_name VARCHAR(50) NOT NULL, UNIQUE(pr_id, label_name))
- **code_diffs** (diff_id UUID PK NOT NULL, commit_id UUID FK NOT NULL, pr_id UUID FK NOT NULL, file_path NOT NULL, change_type NOT NULL, additions NOT NULL, deletions NOT NULL, diff_content TEXT NULL, created_at NOT NULL)
- **files** (file_id UUID PK NOT NULL, repository_id UUID FK NOT NULL, file_path NOT NULL, file_type NULL, status NOT NULL, created_at TIMESTAMP NOT NULL, updated_at NOT NULL, UNIQUE(repository_id, file_path))
- **code_entities** (entity_id UUID PK NOT NULL, file_id UUID FK NOT NULL, entity_name NOT NULL, entity_type VARCHAR(50) NOT NULL, signature NULL, doc_comment NULL, start_line NOT NULL, end_line NOT NULL, created_at NOT NULL, updated_at NOT NULL)
- **dependencies** (dependency_id UUID PK NOT NULL, source_entity_id UUID FK NOT NULL, target_entity_id UUID FK NOT NULL, dependency_type NOT NULL, created_at NOT NULL, UNIQUE(source_entity_id, target_entity_id, dependency_type))

3.3.2 Báo cáo Chứng minh Chuẩn hóa 3NF

Khai báo Tập Phụ thuộc Hàm (Functional Dependencies - FDs)

Xét bảng **repositories** có tập phụ thuộc hàm $F = \{f_1, f_2\}$ như sau:

- f_1 : repository_id $\rightarrow$ workspace_id, name, url, default_branch, sync_status, last_synced_at, created_at, updated_at
- f_2 : url $\rightarrow$ repository_id (Khóa ứng viên duy nhất định danh đường dẫn kho chứa trên VCS)

Xét bảng **commits** có tập phụ thuộc hàm:

3.3. LƯỢC ĐỒ QUAN HỆ LÔ-GÍC (LOGICAL SCHEMA) & 3NF-PHÂN HỆ MÃ NGUỒN & CẤU TRÚC C

- f_3 : `commit_id` $\rightarrow$ `branch_id`, `user_id`, `commit_hash`, `message`, `author_name`, `author_email`, `committed_at`, `created_at`
- f_4 : `commit_hash` $\rightarrow$ `commit_id` (Khóa ứng viên duy nhất từ Git Hash)

Đánh giá Cấp Chuẩn

1. Đạt Chuẩn 1NF (First Normal Form):

- Tất cả các cột trong các bảng của phân hệ đều chứa giá trị nguyên tố (Atomic Value).
- Thuộc tính phức hợp *AuthorInfo* (của Commit) đã được phẳng hóa thành `author_name` và `author_email`.
- Thuộc tính phức hợp *position* (của CodeEntity) đã được phẳng hóa thành `start_line` và `end_line`.
- Thuộc tính đa trị *labels* (của PullRequest) đã được tách triệt để sang bảng phụ `pull_request_labels`.

2. Đạt Chuẩn 2NF (Second Normal Form):

- Các bảng trong lược đồ đều sử dụng Khóa chính đơn (Surrogate PK dạng UUID như `repository_id`, `commit_id`, `pr_id`,...), không sử dụng khóa chính phức hợp cho các thực thể gốc.
- Do đó, mọi thuộc tính không khóa đều phụ thuộc hoàn toàn và trực tiếp vào toàn bộ khóa chính, hoàn toàn không tồn tại phụ thuộc một phần (Partial Dependency).

3. Đạt Chuẩn 3NF (Third Normal Form):

- Mọi thuộc tính không khóa trong các bảng đều phụ thuộc trực tiếp vào Khóa chính (ví dụ trong bảng `commits`, các thông tin như `message`, `author_name`, `committed_at` phụ thuộc trực tiếp vào `commit_id`).
- Không tồn tại phụ thuộc bắc cầu $X \rightarrow Y \rightarrow Z$ giữa các thuộc tính không khóa.

Kết luận: Lược đồ quan hệ phân hệ VCS & Code Structure đạt tiêu chuẩn **3NF**, đảm bảo tính toàn vẹn dữ liệu, loại bỏ các dị thường (anomalies) khi thêm, sửa, xóa và tối ưu hóa hiệu suất lưu trữ cho hệ thống.

3.4 Lược đồ Quan hệ Lô-gíc (Logical Schema) & 3NF- Phân hệ Mẫu & Tài liệu Cốt lõi

Bảng templates

Tên cột	Khóa	Ràng buộc
template_id	PK	not null
workspace_id	FK	not null
name		not null
description		null
content_structure		not null
creator_type		not null
status		not null
created_at		not null
updated_at		not null

Bảng template_sections

Tên cột	Khóa	Ràng buộc
section_id	PK	not null
template_id	FK	not null
heading_title		not null
order_index		not null
prompt_instruction		null
created_at		not null
updated_at		not null
Ràng buộc bổ sung: UNIQUE(template_id, order_index)		

3.4. LƯỢC ĐỒ QUAN HỆ LÔ-GÍC (LOGICAL SCHEMA) & 3NF- PHÂN HỆ MẪU & TÀI LIỆU CỐT LÕI

Bảng placeholders

Tên cột	Khóa	Ràng buộc
placeholder_id	PK	not null
section_id	FK	not null
variable_name		not null
expected_data_type		not null
description		null
is_required		not null
created_at		not null
updated_at		not null
Ràng buộc bổ sung: UNIQUE(section_id, variable_name)		

Bảng template_mappings

Tên cột	Khóa	Ràng buộc
mapping_id	PK	not null
placeholder_id	FK	not null
entity_id	FK	not null
extraction_rule		null
is_active		not null
Ràng buộc bổ sung: UNIQUE(placeholder_id, entity_id)		

Bảng documents

Tên cột	Khóa	Ràng buộc
document_id	PK	not null
workspace_id	FK	not null
user_id	FK	not null
title		not null
document_type		not null
status		not null
created_at		not null
updated_at		not null
deleted_at		null

Bảng document_versions

Tên cột	Khóa	Ràng buộc
version_id	PK	not null
document_id	FK	not null
version_number		not null
content		not null
change_summary		null
status		not null
ticketed_codes		null
created_at		not null
updated_at		not null
Ràng buộc bổ sung: UNIQUE(document_id, version_number)		

Bảng change_logs

Tên cột	Khóa	Ràng buộc
log_id	PK	not null
version_id	FK	not null
change_type		not null
change_description		not null
diff_content		null
created_at		not null
updated_at		not null

3.4.1 Lược đồ dạng cú pháp rút gọn

- templates(template_id PK, workspace_id FK, name, description, content_structure, creator_type, status, created_at, updated_at)
- template_sections(section_id PK, template_id FK, heading_title, order_index, prompt_instruction, created_at, updated_at, UNIQUE(template_id, order_index))
- placeholders(placeholder_id PK, section_id FK, variable_name, expected_data_type, description, is_required, created_at, updated_at, UNIQUE(section_id, variable_name))
- template_mappings(mapping_id PK, placeholder_id FK, entity_id FK, extraction_rule, is_active, UNIQUE(placeholder_id, entity_id))

3.4. LƯỢC ĐỒ QUAN HỆ LÔ-GÍC (LOGICAL SCHEMA) & 3NF- PHÂN HỆ MẪU & TÀI LIỆU CỐT LÕI

- documents(document_id PK, workspace_id FK, user_id FK, title, document_type, status, created_at, updated_at, deleted_at)
- document_versions(version_id PK, document_id FK, version_number, content, change_summary, status, ticketed_codes, created_at, updated_at, UNIQUE(document_id, version_number))
- change_logs(log_id PK, version_id FK, change_type, change_description, diff_content, created_at, updated_at)

3.4.2 Báo cáo Chứng minh Chuẩn hóa 3NF

Khai báo Tập Phụ thuộc Hàm (Functional Dependencies – FDs)

Xét bảng templates có tập phụ thuộc hàm như sau:

template_id $\rightarrow$ {workspace_id, name, description, content_structure, creator_type, status, created_at, updated_at}

Xét bảng template_sections có tập phụ thuộc hàm:

section_id $\rightarrow$ {template_id, heading_title, order_index, prompt_instruction, created_at, updated_at}

Ngoài ra:

(template_id, order_index) $\rightarrow$ section_id

vì có ràng buộc UNIQUE(template_id, order_index).

Xét bảng placeholders có tập phụ thuộc hàm:

placeholder_id $\rightarrow$ {section_id, variable_name, expected_data_type, description, is_required, created_at, updated_at}

Ngoài ra:

(section_id, variable_name) $\rightarrow$ placeholder_id

vì có ràng buộc UNIQUE(section_id, variable_name).

Xét bảng template_mappings có tập phụ thuộc hàm:

mapping_id $\rightarrow$ {placeholder_id, entity_id, extraction_rule, is_active}

Ngoài ra:

$(placeholder_id, entity_id) \rightarrow mapping_id$

vì có ràng buộc `UNIQUE(placeholder_id, entity_id)`.

Xét bảng `documents` có tập phụ thuộc hàm:

$document_id \rightarrow \{workspace_id, user_id, title, document_type, status, created_at, updated_at\}$

Xét bảng `document_versions` có tập phụ thuộc hàm:

$version_id \rightarrow \{document_id, version_number, content, change_summary, status, ticketed_comment\}$

Ngoài ra:

$(document_id, version_number) \rightarrow version_id$

vì có ràng buộc `UNIQUE(document_id, version_number)`.

Xét bảng `change_logs` có tập phụ thuộc hàm:

$log_id \rightarrow \{version_id, change_type, change_description, diff_content, created_at, updated_at\}$

Đánh giá Cấp Chuẩn

1. Đạt Chuẩn 1NF (First Normal Form)

- Tất cả các cột trong các bảng của phân hệ đều chứa giá trị nguyên tố (Atomic Value).
- Các thuộc tính được lưu trữ dưới dạng đơn trị, không có nhóm thuộc tính lặp hay thuộc tính đa trị trong cùng một ô.
- Các thông tin có tính chất riêng biệt đã được tách thành các bảng tương ứng. Ví dụ, thông tin về Section được lưu trong `template_sections`, thông tin Placeholder được lưu trong `placeholders`, còn thông tin phiên bản tài liệu được lưu trong `document_versions`.

3.4. LƯỢC ĐỒ QUAN HỆ LÔ-GÍC (LOGICAL SCHEMA) & 3NF- PHÂN HỆ MẪU & TÀI LIỆU CỐT LÕI

2. Đạt Chuẩn 2NF (Second Normal Form)

- Các bảng trong lược đồ đều sử dụng Khóa chính đơn (Surrogate PK) như `template_id`, `section_id`, `placeholder_id`, `mapping_id`, `document_id`, `version_id`, `log_id`, không sử dụng khóa chính phức hợp cho các bảng.
- Do khóa chính của các bảng chỉ gồm một thuộc tính, không tồn tại trường hợp thuộc tính không khóa chỉ phụ thuộc vào một phần của khóa chính.
- Vì vậy, không tồn tại phụ thuộc bộ phận (Partial Dependency).
- Các ràng buộc UNIQUE như `UNIQUE(template_id, order_index)`, `UNIQUE(section_id, variable_name)`, `UNIQUE(placeholder_id, entity_id)` và `UNIQUE(document_id, version_number)` được sử dụng để đảm bảo tính duy nhất của dữ liệu, nhưng không làm phát sinh phụ thuộc bộ phận đối với khóa chính đơn.

3. Đạt Chuẩn 3NF (Third Normal Form)

- Mọi thuộc tính không khóa trong các bảng đều phụ thuộc trực tiếp vào Khóa chính của bảng.
- Ví dụ trong bảng `templates`, các thông tin như `workspace_id`, `name`, `description`, `content_structure`, `creator_type`, `status`, `created_at`, `updated_at` đều phụ thuộc trực tiếp vào `template_id`.
- Trong bảng `document_versions`, các thông tin như `document_id`, `version_number`, `content`, `change_summary`, `status`, `ticketed_codes`, `created_at`, `updated_at` phụ thuộc trực tiếp vào `version_id`.
- Tương tự, trong bảng `change_logs`, các thuộc tính `version_id`, `change_type`, `change_description`, `diff_content`, `created_at`, `updated_at` phụ thuộc trực tiếp vào `log_id`.
- Không tồn tại phụ thuộc bắc cầu (Transitive Dependency) giữa các thuộc tính không khóa trong phạm vi các phụ thuộc hàm được xác định.

3.5 Lược đồ Quan hệ Lô-gíc(Logical Schema) & 3NF - Phân hệ minh chứng AI & Giám sát

Bảng review

Tên cột	Khóa	Ràng buộc
review_id	PK	not null
version_id	FK	not null
user_id	FK	not null
status		not null
decision		null
summary_notes		null
created_at		not null
updated_at		not null

UNIQUE (version_id, user_id) Một người dùng chỉ được phép có tối đa 1 bản review cho mỗi phiên bản

Bảng review_comments

Tên cột	Khóa	Ràng buộc
comment_id	PK	not null
review_id	FK	not null
content		not null
is_resolved		not null, default false
created_at		not null
update_at		not null

Bảng grounding_evidences

Tên cột	Khóa	Ràng buộc
evidence_id	PK	not null
version_id	FK	not null
snippet_content		not null
purpose		null
created_at		not null

3.5. LƯỢC ĐỒ QUAN HỆ LÔ-GÍC(LOGICAL SCHEMA) & 3NF - PHÂN HỆ MINH CHỨNG AI & GIÁM S

Bảng approvals

Tên cột	Khóa	Ràng buộc
approval_id	PK	not null
version_id	FK	not null
user_id	FK	not null
decision		not null
rejection_reason		null
approved_at		null

UNIQUE (version_id, user_id) Một người dùng chỉ nên có tối đa 1 bản ghi phê duyệt cho mỗi phiên bản.

Bảng drift_alerts

Tên cột	Khóa	Ràng buộc
alert_id	PK	not null
evidence_id	FK	not null
drift_type		not null
status		not null
detected_at		not null

UNIQUE (evidence_id, drift_type) Một loại drift chỉ nên có tối đa 1 cảnh báo cho mỗi bằng chứng.

Bảng grounding_evidence_code_entities

Tên cột	Khóa	Ràng buộc
link_id	PK	not null
evidence_id	FK	not null
entity_id	FK	not null
similarity_score		not null
match_type		not null

UNIQUE (evidence_id, entity_id) Một bằng chứng chỉ nên liên kết với một thực thể duy nhất.

3.5.1 Lược đồ dạng cú pháp rút gọn

- review(review_id UUID PK, version_id UUID FK, user_id FK, status, decision, summary_notes, created_at, updated_at, UNIQUE (version_id, user_id))
- review_comments(comment_id UUID PK, review_id UUID FK, content, is_resolved, created_at, update_at)
- grounding_evidences(evidence_id UUID PK, version_id UUID FK, snippet_content, purpose, created_at)
- approvals(approval_id UUID PK, version_id UUID FK, user_id UUID FK, decision, rejection_reason, approved_at, UNIQUE (version_id, user_id))
- drift_alerts(alert_id UUID PK, evidence_id UUID FK, drift_type, status, detected_at, UNIQUE (evidence_id, drift_type))
- grounding_evidence_code_entities(link_id UUID PK, evidence_id UUID FK, entity_id UUID FK, similarity_score, match_type, UNIQUE (evidence_id, entity_id))

3.5.2 Báo cáo Chứng minh Chuẩn hóa 3NF**Khai báo Tập Phụ thuộc Hàm (Functional Dependencies – FDs)**

Xét bảng review có tập phụ thuộc hàm như sau:

$\text{review_id} \rightarrow \{\text{version_id}, \text{user_id}, \text{status}, \text{decision}, \text{summary_notes}, \text{created_at}, \text{updated_at}\}$

Xét bảng review_comments có tập phụ thuộc hàm:

$\text{comment_id} \rightarrow \{\text{review_id}, \text{content}, \text{is_resolved}, \text{created_at}, \text{update_at}\}$

Xét bảng grounding_evidences có tập phụ thuộc hàm:

$\text{evidence_id} \rightarrow \{\text{version_id}, \text{snippet_content}, \text{purpose}, \text{created_at}\}$

Xét bảng approvals có tập phụ thuộc hàm:

3.5. LƯỢC ĐỒ QUAN HỆ LÔ-GÍC (LOGICAL SCHEMA) & 3NF - PHÂN HỆ MINH CHỨNG AI & GIÁM S

`approval_id` $\rightarrow$ {`version_id`, `user_id`, `decision`, `rejection_reason`, `approved_at`}

Xét bảng `drift_alerts` có tập phụ thuộc hàm:

`alert_id` $\rightarrow$ {`evidence_id`, `drift_type`, `status`, `detected_at`}

Xét bảng `grounding_evidence_code_entities` có tập phụ thuộc hàm:

`link_id` $\rightarrow$ {`evidence_id`, `entity_id`, `similarity_score`, `match_type`}

Đánh giá Cấp Chuẩn

1. Đạt Chuẩn 1NF (First Normal Form)

- Tất cả các cột trong các bảng của phân hệ đều chứa giá trị nguyên tố (Atomic Value).
- Không có thuộc tính phức hợp hay đa trị. Các trường văn bản như `snippet_content` (của `grounding_evidences`) hoặc `summary_notes` (của `review`) đều lưu trữ dữ liệu dạng phẳng, đơn trị.

2. Đạt Chuẩn 2NF (Second Normal Form)

- Các bảng trong lược đồ đều sử dụng Khóa chính đơn (Surrogate PK như `review_id`, `comment_id`, `evidence_id`, `approval_id`, `alert_id`, `link_id`), không sử dụng khóa chính phức hợp cho các thực thể gốc.
- Do đó, mọi thuộc tính không khóa đều phụ thuộc hoàn toàn và trực tiếp vào toàn bộ khóa chính, hoàn toàn không tồn tại phụ thuộc một phần (Partial Dependency).

3. Đạt Chuẩn 3NF (Third Normal Form)

- Mọi thuộc tính không khóa trong các bảng đều phụ thuộc trực tiếp vào Khóa chính. Ví dụ trong bảng `review`, các thông tin như `status`, `decision`, `created_at` phụ thuộc trực tiếp vào `review_id`.
- Không tồn tại phụ thuộc bắc cầu giữa các thuộc tính không khóa.

Chương 4

Tổng kết Lược đồ Quan hệ Lô-gíc-Chuẩn hóa 3NF (Stage 2)

Sau khi áp dụng thành công **Thuật toán 2.2** (Ánh xạ mô hình ERD Quan niệm sang Lược đồ Quan hệ), toàn bộ thực thể, thuộc tính và mối kết hợp trong hệ thống đã được chuyển đổi hoàn chỉnh sang **Lược đồ ERD ở mức Lô-gíc (Logical ERD)**.

Đầu ra đạt được sau bước này bao gồm:

- **Cấu trúc bảng chuẩn hóa:** Chuyển đổi toàn bộ thực thể và các mối kết hợp $M : N$ thành **31 bảng** dữ liệu độc lập với tên gọi tuân thủ chuẩn `snake_case`.
- **Xử lý thuộc tính phức hợp & đa trị:** Giải quyết triệt để các thuộc tính phức hợp, thuộc tính đa trị và chuyển đổi các mối kết hợp 3 ngôi (như cụm USER-WORKSPACE-ROLE) thành các thực thể liên kết hợp lệ.
- **Cơ chế Khóa chính & Khóa ngoại:** Đã đồng bộ hóa việc sử dụng Khóa chính đơn *Surrogate PK (UUID)* cho tất cả các bảng, đồng thời thiết lập mạng lưới Khóa ngoại (*FK*) chặt chẽ để duy trì ràng buộc toàn vẹn tham chiếu.
- **Đạt dạng chuẩn 3NF/BCNF:** Xác định tập phụ thuộc hàm (FDs) cho 4 phân hệ nghiệp vụ, chứng minh toàn bộ các bảng loại bỏ được rủi ro dư thừa dữ liệu và bất thường thao tác.

76 CHƯƠNG 4. TỔNG KẾT LƯỢC ĐỒ QUAN HỆ LÔ-GÍC-CHUẨN HÓA 3NF (STAGE 2)

Chi tiết sơ đồ ERD mức Lô-gíc và mô tả cấu trúc các bảng được trình bày ngay dưới đây:

Sau khi áp dụng thành công **Thuật toán 2.2** (Ánh xạ mô hình ERD Quan niệm sang Lược đồ Quan hệ), toàn bộ thực thể, thuộc tính và mối kết hợp trong hệ thống đã được chuyển đổi hoàn chỉnh sang **Lược đồ ERD ở mức Lô-gíc (Logical ERD)**.

Đầu ra đạt được sau bước này bao gồm:

- **Cấu trúc bảng chuẩn hóa:** Chuyển đổi toàn bộ thực thể và các mối kết hợp $M : N$ thành **30 bảng** dữ liệu độc lập với tên gọi tuân thủ chuẩn `snake_case`.
- **Xử lý thuộc tính phức hợp & đa trị:** Giải quyết triệt để các thuộc tính phức hợp, thuộc tính đa trị và chuyển đổi các mối kết hợp 3 ngôi (như cụm `USER-WORKSPACE-ROLE`) thành các thực thể liên kết hợp lệ.
- **Cơ chế Khóa chính & Khóa ngoại:** Đã đồng bộ hóa việc sử dụng Khóa chính đơn *Surrogate PK (UUID)* cho tất cả các bảng, đồng thời thiết lập mạng lưới Khóa ngoại (*FK*) chặt chẽ để duy trì ràng buộc toàn vẹn tham chiếu.
- **Đạt dạng chuẩn 3NF/BCNF:** Xác định tập phụ thuộc hàm (FDs) cho 4 phân hệ nghiệp vụ, chứng minh toàn bộ các bảng loại bỏ được rủi ro dư thừa dữ liệu và bất thường thao tác.

Chi tiết sơ đồ ERD mức Lô-gíc và mô tả cấu trúc các bảng được trình bày ngay dưới đây:

Phần III

Thiết Kế Vật Lý

Chương 5

Thiết Kế CSDL Vật Lý & DDL SQL Theo Phân Hệ

5.1 Ánh Xạ Lược Đồ ER Phân Hệ Quản Trị Xác Thực & Phân Quyền (Auth & RBAC Domain)

5.1.1 Cú pháp Rút gọn (Logical Schema Syntax)

- `users` (`user_id` UUID PK, `email` VARCHAR(255), `password_hash` VARCHAR(255), `first_name` VARCHAR(100), `last_name` VARCHAR(100), `status` VARCHAR(20), `created_at` TIMESTAMP, `updated_at` TIMESTAMP, `deleted_at` TIMESTAMP)[cite: 1]
- `workspaces` (`workspace_id` UUID PK, `name` VARCHAR(150), `description` TEXT, `status` VARCHAR(20), `created_at` TIMESTAMP, `updated_at` TIMESTAMP, `deleted_at` TIMESTAMP)[cite: 1]
- `roles` (`role_id` UUID PK, `role_name` VARCHAR(50), `description` TEXT, `created_at` TIMESTAMP, `updated_at` TIMESTAMP)[cite: 1]
- `permissions` (`permission_id` UUID PK, `permission_code` VARCHAR(50), `description` TEXT, `created_at` TIMESTAMP)[cite: 1]
- `github_connections` (`connection_id` UUID PK, `user_id` UUID FK, `github_user_id` VARCHAR(50), `github_username` VARCHAR(100),

82CHƯƠNG 5. THIẾT KẾ CSDL VẬT LÝ & DDL SQL THEO PHÂN HỆ

access_token VARCHAR(500), created_at TIMESTAMP, updated_at TIMESTAMP)[cite: 1]

- user_notification_preferences (preference_id UUID PK, user_id UUID FK, channel VARCHAR(20))[cite: 1]
- github_connection_scopes (scope_id UUID PK, connection_id UUID FK, scope_value VARCHAR(50))[cite: 1]
- workspace_members (membership_id UUID PK, user_id UUID FK, workspace_id UUID FK, role_id UUID FK, joined_at TIMESTAMP, status VARCHAR(20))[cite: 1]
- role_permissions (role_permission_id UUID PK, role_id UUID FK, permission_id UUID FK, granted_at TIMESTAMP)[cite: 1]

5.1.2 Mã DDL SQL Hoàn Chỉnh (schema_member_1.sql)

```
1
2 CREATE TABLE users (
3     user_id          UUID          NOT NULL DEFAULT
4     gen_random_uuid(),
5     email            VARCHAR(255)   NOT NULL ,
6     password_hash    VARCHAR(255)   NULL ,
7     first_name       VARCHAR(100)   NULL ,
8     last_name        VARCHAR(100)   NULL ,
9     status           VARCHAR(20)    NOT NULL ,
10    created_at        TIMESTAMP      NOT NULL DEFAULT
11    CURRENT_TIMESTAMP ,
12    updated_at        TIMESTAMP      NOT NULL DEFAULT
13    CURRENT_TIMESTAMP ,
14    deleted_at        TIMESTAMP      NULL ,
15    CONSTRAINT pk_users PRIMARY KEY (user_id),
16    CONSTRAINT uq_users_email UNIQUE (email)
17 );
18
19 CREATE TABLE workspaces (
20     workspace_id      UUID          NOT NULL DEFAULT
21     gen_random_uuid(),
22     name              VARCHAR(150)   NOT NULL ,
23     description       TEXT           NULL ,
24     status            VARCHAR(20)    NOT NULL ,
25     created_at        TIMESTAMP      NOT NULL DEFAULT
26     CURRENT_TIMESTAMP ,
27     updated_at        TIMESTAMP      NOT NULL DEFAULT
28     CURRENT_TIMESTAMP ,
```

5.1. ÁNH XẠ LƯỢC ĐỒ ER PHÂN HỆ QUẢN TRỊ XÁC THỰC & PHÂN QUYỀN (AUTH & RBAC DOMAIN)

```
24     deleted_at      TIMESTAMP      NULL,
25     CONSTRAINT pk_workspaces PRIMARY KEY (workspace_id)
26 );
27
28
29 CREATE TABLE roles (
30     role_id          UUID            NOT NULL DEFAULT
31     gen_random_uuid(),
32     role_name         VARCHAR(50)     NOT NULL,
33     description       TEXT           NULL,
34     created_at        TIMESTAMP      NOT NULL DEFAULT
35     CURRENT_TIMESTAMP,
36     updated_at        TIMESTAMP      NOT NULL DEFAULT
37     CURRENT_TIMESTAMP,
38     CONSTRAINT pk_roles PRIMARY KEY (role_id)
39 );
40
41
42 CREATE TABLE permissions (
43     permission_id     UUID            NOT NULL DEFAULT
44     gen_random_uuid(),
45     permission_code    VARCHAR(50)     NOT NULL,
46     description       TEXT           NULL,
47     created_at        TIMESTAMP      NOT NULL DEFAULT
48     CURRENT_TIMESTAMP,
49     CONSTRAINT pk_permissions PRIMARY KEY (permission_id),
50     CONSTRAINT uq_permissions_code UNIQUE (permission_code)
51 );
52
53
54 CREATE TABLE github_connections (
55     connection_id     UUID            NOT NULL DEFAULT
56     gen_random_uuid(),
57     user_id           UUID            NOT NULL,
58     github_user_id     VARCHAR(50)     NOT NULL,
59     github_username    VARCHAR(100)    NOT NULL,
60     access_token       VARCHAR(500)    NOT NULL,
61     created_at        TIMESTAMP      NOT NULL DEFAULT
62     CURRENT_TIMESTAMP,
63     updated_at        TIMESTAMP      NOT NULL DEFAULT
64     CURRENT_TIMESTAMP,
65     CONSTRAINT pk_github_connections PRIMARY KEY
66     (connection_id),
67     CONSTRAINT uq_github_connections_user UNIQUE (user_id),
68     CONSTRAINT fk_github_connections_user
69     FOREIGN KEY (user_id) REFERENCES users (user_id)
70     ON UPDATE CASCADE
71     ON DELETE CASCADE
72 );
```

84CHƯƠNG 5. THIẾT KẾ CSDL VẬT LÝ & DDL SQL THEO PHÂN HỆ

```
64
65
66 CREATE TABLE user_notification_preferences (
67     preference_id    UUID                NOT NULL DEFAULT
        gen_random_uuid(),
68     user_id          UUID                NOT NULL ,
69     channel          VARCHAR(20)         NOT NULL ,
70     CONSTRAINT pk_user_notification_preferences PRIMARY KEY
        (preference_id),
71     CONSTRAINT fk_user_notification_pref_user
72         FOREIGN KEY (user_id) REFERENCES users (user_id)
73         ON UPDATE CASCADE
74         ON DELETE CASCADE
75 );
76
77
78 CREATE TABLE github_connection_scopes (
79     scope_id         UUID                NOT NULL DEFAULT
        gen_random_uuid(),
80     connection_id    UUID                NOT NULL ,
81     scope_value      VARCHAR(50)         NOT NULL ,
82     CONSTRAINT pk_github_connection_scopes PRIMARY KEY
        (scope_id),
83     CONSTRAINT fk_github_connection_scopes_conn
84         FOREIGN KEY (connection_id) REFERENCES
        github_connections (connection_id)
85         ON UPDATE CASCADE
86         ON DELETE CASCADE
87 );
88
89 CREATE TABLE workspace_members (
90     membership_id    UUID                NOT NULL DEFAULT
        gen_random_uuid(),
91     user_id          UUID                NOT NULL ,
92     workspace_id     UUID                NOT NULL ,
93     role_id          UUID                NOT NULL ,
94     joined_at        TIMESTAMP           NOT NULL DEFAULT
        CURRENT_TIMESTAMP ,
95     status            VARCHAR(20)        NOT NULL ,
96     CONSTRAINT pk_workspace_members PRIMARY KEY
        (membership_id),
97
98     CONSTRAINT uq_workspace_members_user_workspace UNIQUE
        (user_id, workspace_id),
99     CONSTRAINT fk_workspace_members_user
100         FOREIGN KEY (user_id) REFERENCES users (user_id)
101         ON UPDATE CASCADE
102         ON DELETE CASCADE ,
103     CONSTRAINT fk_workspace_members_workspace
```

5.1. ÁNH XẠ LƯỢC ĐỒ ER PHÂN HỆ QUẢN TRỊ XÁC THỰC & PHÂN QUYỀN (AUTH & RBAC DOMA

```
104         FOREIGN KEY (workspace_id) REFERENCES workspaces
105         (workspace_id)
106         ON UPDATE CASCADE
107         ON DELETE CASCADE,
108     CONSTRAINT fk_workspace_members_role
109     FOREIGN KEY (role_id) REFERENCES roles (role_id)
110     ON UPDATE CASCADE
111     ON DELETE RESTRICT
112 );
113
114 CREATE TABLE role_permissions (
115     role_permission_id UUID          NOT NULL DEFAULT
116     gen_random_uuid(),
117     role_id             UUID          NOT NULL,
118     permission_id       UUID          NOT NULL,
119     granted_at          TIMESTAMP     NOT NULL DEFAULT
120     CURRENT_TIMESTAMP,
121     CONSTRAINT pk_role_permissions PRIMARY KEY
122     (role_permission_id),
123     CONSTRAINT uq_role_permissions_pair UNIQUE (role_id,
124     permission_id),
125     CONSTRAINT fk_role_permissions_role
126     FOREIGN KEY (role_id) REFERENCES roles (role_id)
127     ON UPDATE CASCADE
128     ON DELETE CASCADE,
129     CONSTRAINT fk_role_permissions_permission
130     FOREIGN KEY (permission_id) REFERENCES permissions
131     (permission_id)
132     ON UPDATE CASCADE
133     ON DELETE CASCADE
134 );
135
136 CREATE INDEX idx_github_connections_user_id ON
137     github_connections(user_id);
138 CREATE INDEX idx_workspace_members_workspace_id ON
139     workspace_members(workspace_id);
140 CREATE INDEX idx_workspace_members_user_id ON
141     workspace_members(user_id);
142 CREATE INDEX idx_role_permissions_role_id ON
143     role_permissions(role_id);
144
145 CREATE OR REPLACE FUNCTION set_updated_at()
146 RETURNS TRIGGER AS $$
147 BEGIN
148     NEW.updated_at := CURRENT_TIMESTAMP;
149     RETURN NEW;
150 
```

```

143 END;
144
145
146 CREATE TRIGGER trg_users_updated_at
147     BEFORE UPDATE ON users
148     FOR EACH ROW EXECUTE FUNCTION set_updated_at();
149
150 CREATE TRIGGER trg_workspaces_updated_at
151     BEFORE UPDATE ON workspaces
152     FOR EACH ROW EXECUTE FUNCTION set_updated_at();
153
154 CREATE TRIGGER trg_roles_updated_at
155     BEFORE UPDATE ON roles
156     FOR EACH ROW EXECUTE FUNCTION set_updated_at();
157
158 CREATE TRIGGER trg_github_connections_updated_at
159     BEFORE UPDATE ON github_connections
160     FOR EACH ROW EXECUTE FUNCTION set_updated_at();
161
162 GO

```

5.2 Ảnh Xạ Lược Đồ ER Phân Hệ Không Gian Làm Việc & Tích Hợp VCS (Workspace & External VCS Domain)

5.2.1 Cú pháp Rút gọn (Logical Schema Syntax)

- workspaces (workspace_id UUID PK, name VARCHAR(150), description TEXT, status VARCHAR(20), created_at DATETIMEOFFSET, updated_at DATETIMEOFFSET, deleted_at DATETIMEOFFSET)
- github_connections (connection_id UUID PK, workspace_id UUID FK, provider VARCHAR(50), account_name VARCHAR(100), access_token TEXT, sync_status VARCHAR(20), created_at DATETIME2, updated_at DATETIME2)
- github_connection_scopes (scope_id UUID PK, connection_id UUID FK, scope_value VARCHAR(100), created_at DATETIME2)
- repositories (repository_id UUID PK, workspace_id UUID FK, name VARCHAR(150), url VARCHAR(255), default_branch VARCHAR(50), created_at DATETIME2, updated_at DATETIME2)

5.2. ẢNH XẠ LƯỢC ĐỒ ER PHÂN HỆ KHÔNG GIAN LÀM VIỆC & TÍCH HỢP VCS (WORKSPACE & E

- branches (branch_id UUID PK, repository_id UUID FK, name VARCHAR(100), is_protected BOOLEAN, created_at DATETIME2)

5.2.2 Mã DDL SQL Hoàn Chính (schema_member_workspace.sql)

```
1  -- 1. BANG WORKSPACES (Quan ly khong gian lam viec cua nguoi
   dung)
2  CREATE TABLE workspaces (
3      workspace_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
4      name VARCHAR(150) NOT NULL,
5      description TEXT NULL,
6      status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
7      created_at DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET(),
8      updated_at DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET(),
9      deleted_at DATETIMEOFFSET NULL,
10
11     -- Constraints
12     CONSTRAINT uk_workspace_id_name UNIQUE (workspace_id, name)
13     ,
14     CONSTRAINT chk_workspace_status CHECK (status IN ('ACTIVE',
15     'ARCHIVED'))
16 );
17
18 -- 2. BANG GITHUB_CONNECTIONS (Ket noi tai khoan GitHub voi
   Workspace)
19 CREATE TABLE github_connections (
20     connection_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
21     workspace_id UNIQUEIDENTIFIER NOT NULL,
22     provider VARCHAR(50) NOT NULL DEFAULT 'GITHUB',
23     account_name VARCHAR(100) NOT NULL,
24     access_token TEXT NOT NULL,
25     sync_status VARCHAR(20) NOT NULL DEFAULT 'IDLE',
26     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
27     updated_at DATETIME2 NULL,
28
29     -- Constraints
30     CONSTRAINT uk_github_connection UNIQUE (connection_id,
31     provider),
32     CONSTRAINT chk_sync_status CHECK (sync_status IN ('IDLE', '
33     SYNCING', 'SUCCESS', 'FAILED'))
34 );
35
36 -- 3. BANG GITHUB_CONNECTION_SCOPES (Pham vi quyen han cua ket
   noi GitHub)
37 CREATE TABLE github_connection_scopes (
38     scope_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
39     connection_id UNIQUEIDENTIFIER NOT NULL,
40     scope_value VARCHAR(100) NOT NULL,
```

88CHƯƠNG 5. THIẾT KẾ CSDL VẬT LÝ & DDL SQL THEO PHÂN HỆ

```
37     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
38
39     CONSTRAINT uk_connection_scope UNIQUE (connection_id,
40 scope_value)
41 );
42
43 -- 4. BANG REPOSITORIES (Kho lưu trữ mã nguồn liên kết)
44 CREATE TABLE repositories (
45     repository_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
46     workspace_id UNIQUEIDENTIFIER NOT NULL,
47     name VARCHAR(150) NOT NULL,
48     url VARCHAR(255) NOT NULL,
49     default_branch VARCHAR(50) DEFAULT 'main',
50     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
51     updated_at DATETIME2 NULL,
52
53     CONSTRAINT uk_repository_name UNIQUE (repository_id, name)
54 );
55
56 -- 5. BANG BRANCHES (Cac nhánh làm việc trong Repository)
57 CREATE TABLE branches (
58     branch_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
59     repository_id UNIQUEIDENTIFIER NOT NULL,
60     name VARCHAR(100) NOT NULL,
61     is_protected BIT NOT NULL DEFAULT 0,
62     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
63
64     CONSTRAINT uk_branch_name UNIQUE (repository_id, name)
65 );
66 GO
67
68 -- =====
69 -- KHAI BAO CAC KHOA NGOAI THAM CHIEU (CROSS-DOMAIN & INTERNAL
70 -- FKs)
71 -- =====
72 ALTER TABLE github_connections ADD CONSTRAINT
73     fk_github_connections_workspace
74     FOREIGN KEY (workspace_id) REFERENCES workspaces(
75 workspace_id) ON DELETE CASCADE;
76
77 ALTER TABLE github_connection_scopes ADD CONSTRAINT
78     fk_github_scopes_connection
79     FOREIGN KEY (connection_id) REFERENCES github_connections(
80 connection_id) ON DELETE CASCADE;
81
82 ALTER TABLE repositories ADD CONSTRAINT
83     fk_repositories_workspace
84     FOREIGN KEY (workspace_id) REFERENCES workspaces(
85 workspace_id) ON DELETE CASCADE;
```

5.3. ẢNH XẠ LƯỢC ĐỒ ER PHÂN HỆ CẤU TRÚC MÃ NGUỒN & PHÂN TÍCH AST (CODE STRUCTURE & AST ANALYSIS DOMAIN)

```
78  
79 ALTER TABLE branches ADD CONSTRAINT fk_branches_repository  
80 FOREIGN KEY (repository_id) REFERENCES repositories(  
    repository_id) ON DELETE CASCADE;  
81 GO
```

5.3 Ảnh Xạ Lược Đồ ER Phân Hệ Cấu Trúc Mã Nguồn & Phân Tích AST (Code Structure & AST Analysis Domain)

5.3.1 Cú pháp Rút gọn (Logical Schema Syntax)

- commits
(commit_id UUID PK,
repository_id UUID FK,
branch_id UUID FK,
pr_id UUID FK,
commit_hash VARCHAR(255),
author_name VARCHAR(255),
author_email VARCHAR(255),
message TEXT,
committed_at TIMESTAMP,
created_at TIMESTAMP,
updated_at TIMESTAMP)
- pull_requests (pr_id UUID PK, repository_id UUID FK,
pr_number INTEGER, title VARCHAR(500), description TEXT,
source_branch VARCHAR(255), target_branch VARCHAR(255),
status VARCHAR(20), created_at TIMESTAMP, updated_at
TIMESTAMP, merged_at TIMESTAMP)
- pull_request_labels (pull_request_label_id UUID PK, pr_id
UUID FK, label_name VARCHAR(100), label_color VARCHAR(20))
- files (file_id UUID PK, repository_id UUID FK, file_path
VARCHAR(1000), file_type VARCHAR(50), status VARCHAR(20),
created_at TIMESTAMP, updated_at TIMESTAMP)

90CHƯƠNG 5. THIẾT KẾ CSDL VẬT LÝ & DDL SQL THEO PHÂN HỆ

- `code_diffs` (`diff_id` UUID PK, `commit_id` UUID FK, `pr_id` UUID FK, `file_id` UUID FK, `file_path` VARCHAR(1000), `change_type` VARCHAR(20), `additions` INTEGER, `deletions` INTEGER, `diff_content` TEXT, `created_at` TIMESTAMP)
- `code_entities` (`code_entity_id` UUID PK, `file_id` UUID FK, `entity_name` VARCHAR(255), `entity_type` VARCHAR(50), `signature` TEXT, `doc_comment` TEXT, `start_line` INTEGER, `end_line` INTEGER, `created_at` TIMESTAMP, `updated_at` TIMESTAMP)
- `dependencies` (`dependency_id` UUID PK, `source_entity_id` UUID FK, `target_entity_id` UUID FK, `dependency_type` VARCHAR(50), `created_at` TIMESTAMP)

5.3.2 Mã DDL SQL Hoàn Chỉnh (schema_member_3.sql)

```
1
2 CREATE TABLE commits (
3     commit_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
4     repository_id UNIQUEIDENTIFIER NOT NULL,
5     branch_id UNIQUEIDENTIFIER NOT NULL,
6     pr_id UNIQUEIDENTIFIER NULL,
7     commit_hash VARCHAR(255) NOT NULL,
8     author_name VARCHAR(255) NULL,
9     author_email VARCHAR(255) NULL,
10    message NVARCHAR(MAX) NULL,
11    committed_at DATETIME2 NOT NULL,
12    created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
13    updated_at DATETIME2 NULL,
14
15    CONSTRAINT uq_commits_repository_hash
16        UNIQUE (repository_id, commit_hash)
17 );
18
19 CREATE INDEX idx_commits_repository_id
20     ON commits(repository_id);
21
22 CREATE INDEX idx_commits_branch_id
23     ON commits(branch_id);
24
25 CREATE INDEX idx_commits_pr_id
26     ON commits(pr_id);
27
28 CREATE INDEX idx_commits_committed_at
29     ON commits(committed_at);
30
```

5.3. ẢNH XẠ LƯỢC ĐỒ ER PHÂN HỆ CẤU TRÚC MÃ NGUỒN & PHÂN TÍCH AST (CODE STRUCTURE)

```
31 CREATE TABLE pull_requests (
32     pr_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
33     repository_id UNIQUEIDENTIFIER NOT NULL,
34     pr_number INT NOT NULL,
35     title VARCHAR(500) NOT NULL,
36     description NVARCHAR(MAX) NULL,
37     source_branch VARCHAR(255) NULL,
38     target_branch VARCHAR(255) NULL,
39     status VARCHAR(20) NOT NULL DEFAULT 'OPEN',
40     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
41     updated_at DATETIME2 NULL,
42     merged_at DATETIME2 NULL,
43
44     CONSTRAINT uq_pull_requests_repository_pr
45         UNIQUE (repository_id, pr_number),
46
47     CONSTRAINT chk_pull_requests_status
48         CHECK (status IN ('OPEN', 'CLOSED', 'MERGED'))
49 );
50
51 CREATE INDEX idx_pull_requests_repository_id
52     ON pull_requests(repository_id);
53
54 CREATE INDEX idx_pull_requests_status
55     ON pull_requests(status);
56
57 CREATE INDEX idx_pull_requests_created_at
58     ON pull_requests(created_at);
59
60 CREATE TABLE pull_request_labels (
61     pull_request_label_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT
62     NEWID(),
63     pr_id UNIQUEIDENTIFIER NOT NULL,
64     label_name VARCHAR(100) NOT NULL,
65     label_color VARCHAR(20) NULL,
66
67     CONSTRAINT uq_pull_request_labels
68         UNIQUE (pr_id, label_name)
69 );
70
71 CREATE INDEX idx_pull_request_labels_pr_id
72     ON pull_request_labels(pr_id);
73
74 CREATE INDEX idx_pull_request_labels_name
75     ON pull_request_labels(label_name);
76
77 CREATE TABLE files (
78     file_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
79     repository_id UNIQUEIDENTIFIER NOT NULL,
```

92CHƯƠNG 5. THIẾT KẾ CSDL VẬT LÝ & DDL SQL THEO PHÂN HỆ

```
79     file_path VARCHAR(1000) NOT NULL ,
80     file_type VARCHAR(50) NULL ,
81     status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
82     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
83     updated_at DATETIME2 NULL ,
84
85     CONSTRAINT uq_files_repository_file_path
86         UNIQUE (repository_id, file_path),
87
88     CONSTRAINT chk_files_status
89         CHECK (status IN ('ACTIVE', 'DELETED', 'RENAMED'))
90 );
91
92 CREATE INDEX idx_files_repository_id
93     ON files(repository_id);
94
95 CREATE INDEX idx_files_file_path
96     ON files(file_path);
97
98 CREATE INDEX idx_files_file_type
99     ON files(file_type);
100
101 CREATE INDEX idx_files_status
102     ON files(status);
103
104 CREATE TABLE code_diffs (
105     diff_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
106     commit_id UNIQUEIDENTIFIER NOT NULL ,
107     pr_id UNIQUEIDENTIFIER NULL ,
108     file_id UNIQUEIDENTIFIER NOT NULL ,
109     file_path VARCHAR(1000) NULL ,
110     change_type VARCHAR(20) NOT NULL ,
111     additions INT NULL ,
112     deletions INT NULL ,
113     diff_content NVARCHAR(MAX) NULL ,
114     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
115
116     CONSTRAINT chk_code_diffs_change_type
117         CHECK (change_type IN (
118             'ADDED',
119             'MODIFIED',
120             'DELETED',
121             'RENAMED'
122         )),
123
124     CONSTRAINT uq_code_diffs_commit_file
125         UNIQUE (commit_id, file_id)
126 );
127
```

5.3. ẢNH XẠ LƯỢC ĐỒ ER PHÂN HỆ CẤU TRÚC MÃ NGUỒN & PHÂN TÍCH AST (CODE STRUCTURE)

```
128 CREATE INDEX idx_code_diffs_commit_id
129     ON code_diffs(commit_id);
130
131 CREATE INDEX idx_code_diffs_pr_id
132     ON code_diffs(pr_id);
133
134 CREATE INDEX idx_code_diffs_file_id
135     ON code_diffs(file_id);
136
137 CREATE INDEX idx_code_diffs_change_type
138     ON code_diffs(change_type);
139
140 CREATE TABLE code_entities (
141     code_entity_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT
142     NEWID(),
143     file_id UNIQUEIDENTIFIER NOT NULL,
144     entity_name VARCHAR(255) NOT NULL,
145     entity_type VARCHAR(50) NOT NULL,
146     signature NVARCHAR(MAX) NULL,
147     doc_comment NVARCHAR(MAX) NULL,
148     start_line INT NULL,
149     end_line INT NULL,
150     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
151     updated_at DATETIME2 NULL,
152
153     CONSTRAINT chk_code_entities_entity_type
154         CHECK (entity_type IN (
155             'CLASS',
156             'INTERFACE',
157             'FUNCTION',
158             'METHOD',
159             'VARIABLE',
160             'CONSTANT',
161             'MODULE',
162             'PACKAGE',
163             'ENUM',
164             'STRUCT'
165         )),
166
167     CONSTRAINT chk_code_entities_line_range
168         CHECK (
169             start_line IS NULL
170             OR end_line IS NULL
171             OR start_line <= end_line
172         ),
173
174     CONSTRAINT uq_code_entities
175         UNIQUE (file_id, entity_name, entity_type, start_line)
176 );
```

94CHƯƠNG 5. THIẾT KẾ CSDL VẬT LÝ & DDL SQL THEO PHÂN HỆ

```
176
177 CREATE INDEX idx_code_entities_file_id
178     ON code_entities(file_id);
179
180 CREATE INDEX idx_code_entities_entity_name
181     ON code_entities(entity_name);
182
183 CREATE INDEX idx_code_entities_entity_type
184     ON code_entities(entity_type);
185
186 CREATE TABLE dependencies (
187     dependency_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
188     source_entity_id UNIQUEIDENTIFIER NOT NULL,
189     target_entity_id UNIQUEIDENTIFIER NOT NULL,
190     dependency_type VARCHAR(50) NOT NULL,
191     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
192
193     CONSTRAINT uq_dependencies
194         UNIQUE (
195             source_entity_id,
196             target_entity_id,
197             dependency_type
198         ),
199
200     CONSTRAINT chk_dependencies_dependency_type
201         CHECK (dependency_type IN (
202             'CALLS',
203             'IMPORTS',
204             'INHERITS',
205             'IMPLEMENTS',
206             'USES',
207             'REFERENCES',
208             'DEPENDS_ON'
209         )),
210
211     CONSTRAINT chk_dependencies_no_self_reference
212         CHECK (source_entity_id <> target_entity_id)
213 );
214
215 CREATE INDEX idx_dependencies_source_entity_id
216     ON dependencies(source_entity_id);
217
218 CREATE INDEX idx_dependencies_target_entity_id
219     ON dependencies(target_entity_id);
220
221 CREATE INDEX idx_dependencies_dependency_type
222     ON dependencies(dependency_type);
223
224 GO
```

5.3. ẢNH XẠ LƯỢC ĐỒ ER PHÂN HỆ CẤU TRÚC MÃ NGUỒN & PHÂN TÍCH AST (CODE STRUCTURE)

```
225
226 ALTER TABLE commits ADD CONSTRAINT fk_commits_repository
227     FOREIGN KEY (repository_id)
228     REFERENCES repositories(repository_id);
229
230 ALTER TABLE commits ADD CONSTRAINT fk_commits_branch
231     FOREIGN KEY (branch_id)
232     REFERENCES branches(branch_id);
233
234 ALTER TABLE commits ADD CONSTRAINT fk_commits_pull_request
235     FOREIGN KEY (pr_id)
236     REFERENCES pull_requests(pr_id)
237     ON DELETE SET NULL;
238
239
240 -- PULL REQUESTS
241
242 ALTER TABLE pull_requests ADD CONSTRAINT
243     fk_pull_requests_repository
244     FOREIGN KEY (repository_id)
245     REFERENCES repositories(repository_id);
246
247 -- PULL REQUEST LABELS
248
249 ALTER TABLE pull_request_labels ADD CONSTRAINT
250     fk_pull_request_labels_pr
251     FOREIGN KEY (pr_id)
252     REFERENCES pull_requests(pr_id)
253     ON DELETE CASCADE;
254
255 -- FILES
256
257 ALTER TABLE files ADD CONSTRAINT fk_files_repository
258     FOREIGN KEY (repository_id)
259     REFERENCES repositories(repository_id)
260     ON DELETE CASCADE;
261
262
263 -- CODE DIFFS
264
265 ALTER TABLE code_diffs ADD CONSTRAINT fk_code_diffs_commit
266     FOREIGN KEY (commit_id)
267     REFERENCES commits(commit_id)
268     ON DELETE CASCADE;
269
270 ALTER TABLE code_diffs ADD CONSTRAINT
271     fk_code_diffs_pull_request
```

```

271     FOREIGN KEY (pr_id)
272     REFERENCES pull_requests(pr_id)
273     ON DELETE SET NULL;
274
275 ALTER TABLE code_diffs ADD CONSTRAINT fk_code_diffs_file
276     FOREIGN KEY (file_id)
277     REFERENCES files(file_id)
278     ON DELETE CASCADE;
279
280
281 -- CODE ENTITIES
282
283 ALTER TABLE code_entities ADD CONSTRAINT fk_code_entities_file
284     FOREIGN KEY (file_id)
285     REFERENCES files(file_id)
286     ON DELETE CASCADE;
287
288
289 -- DEPENDENCIES
290
291 ALTER TABLE dependencies ADD CONSTRAINT
292     fk_dependencies_source_entity
293     FOREIGN KEY (source_entity_id)
294     REFERENCES code_entities(code_entity_id)
295     ON DELETE CASCADE;
296
297 ALTER TABLE dependencies ADD CONSTRAINT
298     fk_dependencies_target_entity
299     FOREIGN KEY (target_entity_id)
300     REFERENCES code_entities(code_entity_id)
301     ON DELETE CASCADE;
302
303 GO

```

5.4 Ánh Xạ Lược Đồ ER Phân Hệ Khuôn Mẫu & Quản Lý Vòng Đời Tài Liệu (Template & Core Documentation Domain)

5.4.1 Cú pháp Rút gọn (Logical Schema Syntax)

- `templates` (template_id UUID PK, workspace_id UUID FK, name VARCHAR(150), description TEXT, content_structure TEXT,

5.4. ÁNH XẠ LƯỢC ĐỒ ER PHÂN HỆ KHUÔN MẪU & QUẢN LÝ VÒNG ĐỜI TÀI LIỆU (TEMPLATE &

creator_type VARCHAR(20), status VARCHAR(20), created_at
TIMESTAMP, updated_at TIMESTAMP)

- template_sections (section_id UUID PK, template_id UUID FK, heading_title VARCHAR(255), order_index INTEGER, prompt_instruction TEXT, created_at TIMESTAMP, updated_at TIMESTAMP)
- placeholders (placeholder_id UUID PK, section_id UUID FK, variable_name VARCHAR(100), expected_data_type VARCHAR(30), description TEXT, is_required BOOLEAN, created_at TIMESTAMP, updated_at TIMESTAMP)
- template_mappings (mapping_id UUID PK, placeholder_id UUID FK, code_entity_id UUID FK, extraction_rule TEXT, is_active BOOLEAN)
- documents (document_id UUID PK, workspace_id UUID FK, author_user_id UUID FK, title VARCHAR(255), document_type VARCHAR(30), status VARCHAR(20), created_at TIMESTAMP, updated_at TIMESTAMP, deleted_at TIMESTAMP)
- document_versions (version_id UUID PK, document_id UUID FK, version_number VARCHAR(20), content TEXT, change_summary TEXT, status VARCHAR(20), ticket_codes VARCHAR(500), created_at TIMESTAMP, updated_at TIMESTAMP)
- change_logs (log_id UUID PK, version_id UUID FK, change_type VARCHAR(20), change_description TEXT, diff_content TEXT, created_at TIMESTAMP, updated_at TIMESTAMP)

5.4.2 Mã DDL SQL Hoàn Chỉnh (schema_member_4.sql)

```
1
2
3
4 CREATE TABLE templates (
5     template_id          UUID          NOT NULL DEFAULT
6     gen_random_uuid(),
7     workspace_id         UUID          NOT NULL,
8     name                 VARCHAR(150)  NOT NULL,
9     description          TEXT          NULL,
10    content_structure     TEXT          NULL,
11    creator_type          VARCHAR(20)   NOT NULL,
```

98CHƯƠNG 5. THIẾT KẾ CSDL VẬT LÝ & DDL SQL THEO PHÂN HỆ

```

11     status          VARCHAR(20)      NOT NULL DEFAULT
    'DRAFT',
12     created_at      TIMESTAMP        NOT NULL DEFAULT
    CURRENT_TIMESTAMP,
13     updated_at      TIMESTAMP        NOT NULL DEFAULT
    CURRENT_TIMESTAMP,
14     CONSTRAINT pk_templates PRIMARY KEY (template_id),
15
16     CONSTRAINT uq_templates_workspace_name UNIQUE
    (workspace_id, name),
17     CONSTRAINT fk_templates_workspace
18     FOREIGN KEY (workspace_id) REFERENCES workspaces
    (workspace_id)
19         ON UPDATE CASCADE
20         ON DELETE CASCADE,
21     CONSTRAINT ck_templates_creator_type
22     CHECK (creator_type IN ('SYSTEM', 'USER', 'AI')),
23     CONSTRAINT ck_templates_status
24     CHECK (status IN ('DRAFT', 'ACTIVE', 'ARCHIVED')),
25     CONSTRAINT ck_templates_name_not_blank
26     CHECK (length(btrim(name)) > 0),
27     CONSTRAINT ck_templates_updated_after_created
28     CHECK (updated_at >= created_at)
29 );
30
31 CREATE TABLE template_sections (
32     section_id      UUID              NOT NULL DEFAULT
    gen_random_uuid(),
33     template_id     UUID              NOT NULL,
34     heading_title   VARCHAR(255)     NOT NULL,
35     order_index     INTEGER           NOT NULL,
36     prompt_instruction TEXT          NULL,
37     created_at      TIMESTAMP        NOT NULL DEFAULT
    CURRENT_TIMESTAMP,
38     updated_at      TIMESTAMP        NOT NULL DEFAULT
    CURRENT_TIMESTAMP,
39     CONSTRAINT pk_template_sections PRIMARY KEY (section_id),
40
41     CONSTRAINT uq_template_sections_order UNIQUE (template_id,
    order_index),
42     CONSTRAINT fk_template_sections_template
43     FOREIGN KEY (template_id) REFERENCES templates
    (template_id)
44         ON UPDATE CASCADE
45         ON DELETE CASCADE,
46     CONSTRAINT ck_template_sections_order_index
47     CHECK (order_index >= 0),
48     CONSTRAINT ck_template_sections_heading_not_blank
49     CHECK (length(btrim(heading_title)) > 0)

```

5.4. ÁNH XẠ LƯỢC ĐỒ ER PHÂN HỆ KHUÔN MẪU & QUẢN LÝ VÒNG ĐỜI TÀI LIỆU (TEMPLATE &

```
50 );
51
52 CREATE TABLE placeholders (
53     placeholder_id        UUID                NOT NULL DEFAULT
gen_random_uuid(),
54     section_id            UUID                NOT NULL,
55     variable_name          VARCHAR(100)        NOT NULL,
56     expected_data_type     VARCHAR(30)        NOT NULL,
57     description            TEXT                NULL,
58     is_required            BOOLEAN             NOT NULL DEFAULT TRUE,
59     created_at             TIMESTAMP          NOT NULL DEFAULT
CURRENT_TIMESTAMP,
60     updated_at             TIMESTAMP          NOT NULL DEFAULT
CURRENT_TIMESTAMP,
61     CONSTRAINT pk_placeholders PRIMARY KEY (placeholder_id),
62
63     CONSTRAINT uq_placeholders_section_variable UNIQUE
(section_id, variable_name),
64     CONSTRAINT fk_placeholders_section
65     FOREIGN KEY (section_id) REFERENCES template_sections
(section_id)
66     ON UPDATE CASCADE
67     ON DELETE CASCADE,
68     CONSTRAINT ck_placeholders_expected_data_type
69     CHECK (expected_data_type IN ('STRING', 'NUMBER',
'BOOLEAN',
70                                     'DATE', 'LIST',
'OBJECT', 'CODE')),
71
72     CONSTRAINT ck_placeholders_variable_name_format
73     CHECK (variable_name ~ '^[A-Za-z_][A-Za-z0-9_]*$')
74 );
75
76 CREATE TABLE template_mappings (
77     mapping_id            UUID                NOT NULL DEFAULT
gen_random_uuid(),
78     placeholder_id        UUID                NOT NULL,
79     code_entity_id        UUID                NOT NULL,
80     extraction_rule        TEXT                NULL,
81     is_active             BOOLEAN             NOT NULL DEFAULT TRUE,
82     CONSTRAINT pk_template_mappings PRIMARY KEY (mapping_id),
83
84     CONSTRAINT uq_template_mappings_pair UNIQUE
(placeholder_id, code_entity_id),
85     CONSTRAINT fk_template_mappings_placeholder
86     FOREIGN KEY (placeholder_id) REFERENCES placeholders
(placeholder_id)
87     ON UPDATE CASCADE
88     ON DELETE CASCADE,
```

```

89     CONSTRAINT fk_template_mappings_code_entity
90     FOREIGN KEY (code_entity_id) REFERENCES code_entities
91     (code_entity_id)
92     ON UPDATE CASCADE
93     ON DELETE CASCADE
94 );
95 CREATE TABLE documents (
96     document_id          UUID          NOT NULL DEFAULT
97     gen_random_uuid(),
98     workspace_id         UUID          NOT NULL,
99     author_user_id       UUID          NOT NULL,
100    title                 VARCHAR(255)  NOT NULL,
101    document_type         VARCHAR(30)    NOT NULL,
102    status                VARCHAR(20)    NOT NULL DEFAULT
103    'DRAFT',
104    created_at            TIMESTAMP      NOT NULL DEFAULT
105    CURRENT_TIMESTAMP,
106    updated_at            TIMESTAMP      NOT NULL DEFAULT
107    CURRENT_TIMESTAMP,
108    deleted_at            TIMESTAMP      NULL,
109    CONSTRAINT pk_documents PRIMARY KEY (document_id),
110    CONSTRAINT fk_documents_workspace
111    FOREIGN KEY (workspace_id) REFERENCES workspaces
112    (workspace_id)
113    ON UPDATE CASCADE
114    ON DELETE CASCADE,
115    CONSTRAINT fk_documents_author
116    FOREIGN KEY (author_user_id) REFERENCES users (user_id)
117    ON UPDATE CASCADE
118    ON DELETE RESTRICT,
119    CONSTRAINT ck_documents_type
120    CHECK (document_type IN ('SRS', 'API_DOC',
121    'USER_GUIDE',
122    'RELEASE_NOTE',
123    'TECH_DESIGN', 'OTHER')),
124    CONSTRAINT ck_documents_status
125    CHECK (status IN ('DRAFT', 'IN_REVIEW', 'PUBLISHED',
126    'ARCHIVED')),
127    CONSTRAINT ck_documents_title_not_blank
128    CHECK (length(btrim(title)) > 0),
129    CONSTRAINT ck_documents_deleted_after_created
130    CHECK (deleted_at IS NULL OR deleted_at >= created_at)
131 );
132
133 CREATE UNIQUE INDEX uq_documents_workspace_title_active
134 ON documents (workspace_id, title)
135 WHERE deleted_at IS NULL;
136

```

5.4. ẢNH XẠ LƯỢC ĐỒ ER PHÂN HỆ KHUÔN MẪU & QUẢN LÝ VÒNG ĐỜI TÀI LIỆU (TEMPLATE &

```
129 CREATE TABLE document_versions (
130     version_id          UUID          NOT NULL DEFAULT
        gen_random_uuid(),
131     document_id         UUID          NOT NULL,
132     version_number      VARCHAR(20)   NOT NULL,
133     content             TEXT          NULL,
134     change_summary      TEXT          NULL,
135     status              VARCHAR(20)   NOT NULL DEFAULT
        'DRAFT',
136     ticket_codes        VARCHAR(500)  NULL,
137     created_at          TIMESTAMP     NOT NULL DEFAULT
        CURRENT_TIMESTAMP,
138     updated_at          TIMESTAMP     NOT NULL DEFAULT
        CURRENT_TIMESTAMP,
139     CONSTRAINT pk_document_versions PRIMARY KEY (version_id),
140
141     CONSTRAINT uq_document_versions_number UNIQUE
        (document_id, version_number),
142     CONSTRAINT fk_document_versions_document
        FOREIGN KEY (document_id) REFERENCES documents
        (document_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
143     CONSTRAINT ck_document_versions_status
        CHECK (status IN ('DRAFT', 'IN_REVIEW', 'APPROVED',
        'REJECTED', 'PUBLISHED')),
144
145     CONSTRAINT ck_document_versions_number_format
        CHECK (version_number ~ '^[vV]?[0-9]+(\.[0-9]+){0,2}$')
151 );
152
153 CREATE TABLE change_logs (
154     log_id              UUID          NOT NULL DEFAULT
        gen_random_uuid(),
155     version_id          UUID          NOT NULL,
156     change_type         VARCHAR(20)   NOT NULL,
157     change_description  TEXT          NULL,
158     diff_content        TEXT          NULL,
159     created_at          TIMESTAMP     NOT NULL DEFAULT
        CURRENT_TIMESTAMP,
160     updated_at          TIMESTAMP     NOT NULL DEFAULT
        CURRENT_TIMESTAMP,
161     CONSTRAINT pk_change_logs PRIMARY KEY (log_id),
162     CONSTRAINT fk_change_logs_version
        FOREIGN KEY (version_id) REFERENCES document_versions
        (version_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
163     CONSTRAINT ck_change_logs_change_type
```

```

167         CHECK (change_type IN ('CREATE', 'UPDATE', 'DELETE',
168                                'RESTORE', 'PUBLISH'))
169     );
170
171     CREATE INDEX idx_templates_workspace_id ON templates
172         (workspace_id);
173     CREATE INDEX idx_templates_status ON templates (status);
174
175     CREATE INDEX idx_template_mappings_code_entity ON
176         template_mappings (code_entity_id);
177     CREATE INDEX idx_template_mappings_active ON
178         template_mappings (placeholder_id)
179         WHERE is_active = TRUE;
180
181     CREATE INDEX idx_documents_workspace_id ON documents
182         (workspace_id);
183     CREATE INDEX idx_documents_author ON documents
184         (author_user_id);
185     CREATE INDEX idx_documents_status ON documents (status);
186     CREATE INDEX idx_documents_created_at ON documents
187         (created_at DESC);
188
189     CREATE INDEX idx_document_versions_status ON
190         document_versions (status);
191     CREATE INDEX idx_document_versions_created_at ON
192         document_versions (document_id, created_at DESC);
193
194     CREATE INDEX idx_change_logs_version_id ON change_logs
195         (version_id);
196     CREATE INDEX idx_change_logs_created_at ON change_logs
197         (created_at DESC);
198
199     CREATE OR REPLACE FUNCTION set_updated_at()
200     RETURNS TRIGGER AS $$
201     BEGIN
202         NEW.updated_at := CURRENT_TIMESTAMP;
203         RETURN NEW;
204     END;
205
206     CREATE TRIGGER trg_templates_updated_at
207         BEFORE UPDATE ON templates
208         FOR EACH ROW EXECUTE FUNCTION set_updated_at();
209     CREATE TRIGGER trg_template_sections_updated_at
210         BEFORE UPDATE ON template_sections
211         FOR EACH ROW EXECUTE FUNCTION set_updated_at();
212     CREATE TRIGGER trg_placeholders_updated_at
213         BEFORE UPDATE ON placeholders
214         FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

5.5. NH XẠ LƯỢC ĐỒ ER PHÂN HỆ KIỂM DUYỆT, BẰNG CHỨNG AI & GIÁM SÁT HỆ THỐNG (GOVERNANCE, AI EVIDENCE & OBSERVABILITY DOMAIN)

```
205 CREATE TRIGGER trg_documents_updated_at
206     BEFORE UPDATE ON documents
207     FOR EACH ROW EXECUTE FUNCTION set_updated_at();
208 CREATE TRIGGER trg_document_versions_updated_at
209     BEFORE UPDATE ON document_versions
210     FOR EACH ROW EXECUTE FUNCTION set_updated_at();
211 CREATE TRIGGER trg_change_logs_updated_at
212     BEFORE UPDATE ON change_logs
213     FOR EACH ROW EXECUTE FUNCTION set_updated_at();
214 GO
```

5.5 nh Xạ Lược Đồ ER Phân Hệ Kiểm Duyệt, Bằng Chứng AI & Giám Sát Hệ Thống (Governance, AI Evidence & Observability Domain)

5.5.1 Cú pháp Rút gọn (Logical Schema Syntax)

- reviews (review_id UUID PK, version_id UUID FK, user_id UUID FK, status VARCHAR(20), decision VARCHAR(20), summary_notes TEXT, created_at TIMESTAMP, updated_at TIMESTAMP)
- review_comments (comment_id UUID PK, review_id UUID FK, user_id UUID FK, content TEXT, is_resolved BOOLEAN, created_at TIMESTAMP, updated_at TIMESTAMP)
- approvals (approval_id UUID PK, version_id UUID FK, user_id UUID FK, decision VARCHAR(20), rejection_reason TEXT, approved_at TIMESTAMP)
- grounding_evidences (evidence_id UUID PK, version_id UUID FK, snippet_content TEXT, purpose VARCHAR(30), created_at TIMESTAMP)
- drift_alerts (alert_id UUID PK, evidence_id UUID FK, repository_id UUID FK, drift_type VARCHAR(20), status VARCHAR(20), detected_at TIMESTAMP)

5.5.2 Mã DDL SQL Hoàn Chỉnh (schema_member_5.sql)

```

1
2
3 CREATE TABLE reviews (
4     review_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
5     version_id UNIQUEIDENTIFIER NOT NULL,
6     user_id UNIQUEIDENTIFIER NOT NULL,
7     status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
8     decision VARCHAR(20) NULL,
9     summary_notes NVARCHAR(MAX) NULL,
10    created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
11    updated_at DATETIME2 NULL,
12
13    CONSTRAINT chk_reviews_status CHECK (status IN ('PENDING',
14    'IN_PROGRESS', 'COMPLETED')),
15    CONSTRAINT chk_reviews_decision CHECK (decision IS NULL OR
16    decision IN ('APPROVED', 'REJECTED', 'CHANGES_REQUESTED'))
17 );
18
19 CREATE TABLE review_comments (
20     comment_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
21     review_id UNIQUEIDENTIFIER NOT NULL,
22     user_id UNIQUEIDENTIFIER NOT NULL,
23     content NVARCHAR(MAX) NOT NULL,
24     is_resolved BIT NOT NULL DEFAULT 0,
25     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
26     updated_at DATETIME2 NULL,
27
28     CONSTRAINT fk_comments_review FOREIGN KEY (review_id)
29     REFERENCES reviews(review_id) ON DELETE CASCADE
30 );
31
32 CREATE TABLE approvals (
33     approval_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
34     version_id UNIQUEIDENTIFIER NOT NULL,
35     user_id UNIQUEIDENTIFIER NOT NULL,
36     decision VARCHAR(20) NOT NULL,
37     rejection_reason NVARCHAR(MAX) NULL,
38     approved_at DATETIME2 NOT NULL DEFAULT GETDATE(),
39
40     CONSTRAINT chk_approvals_decision CHECK (decision IN ('
41     APPROVED', 'REJECTED'))
42 );
43
44 CREATE TABLE grounding_evidences (
45     evidence_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
46     version_id UNIQUEIDENTIFIER NOT NULL,
47     snippet_content NVARCHAR(MAX) NOT NULL,
48     purpose VARCHAR(30) NOT NULL DEFAULT '
49     HALLUCINATION_PREVENTION',

```

5.5. NH XẠ LƯỢC ĐỒ ER PHÂN HỆ KIỂM DUYỆT, BẢNG CHỨNG AI & GIÁM SÁT HỆ THỐNG (GOV

```
46     created_at DATETIME2 NOT NULL DEFAULT GETDATE(),
47
48     CONSTRAINT chk_evidences_purpose CHECK (purpose IN ('
HALLUCINATION_PREVENTION', 'CODE_REFERENCE', '
AST_VERIFICATION'))
49 );
50
51 CREATE TABLE drift_alerts (
52     alert_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
53     evidence_id UNIQUEIDENTIFIER NOT NULL,
54     repository_id UNIQUEIDENTIFIER NOT NULL,
55     drift_type VARCHAR(20) NOT NULL,
56     status VARCHAR(20) NOT NULL DEFAULT 'OPEN',
57     detected_at DATETIME2 NOT NULL DEFAULT GETDATE(),
58
59     CONSTRAINT fk_alerts_evidence FOREIGN KEY (evidence_id)
60     REFERENCES grounding_evidences(evidence_id) ON DELETE
CASCADE,
61     CONSTRAINT chk_alerts_type CHECK (drift_type IN ('
REFERENTIAL', 'SIGNATURE', 'SEMANTIC', 'OUTDATED')),
62     CONSTRAINT chk_alerts_status CHECK (status IN ('OPEN', '
RESOLVED', 'IGNORED'))
63 );
64
65
66
67 ALTER TABLE reviews ADD CONSTRAINT fk_reviews_version
68     FOREIGN KEY (version_id) REFERENCES document_versions(
version_id);
69 ALTER TABLE reviews ADD CONSTRAINT fk_reviews_user
70     FOREIGN KEY (user_id) REFERENCES users(user_id);
71 ALTER TABLE review_comments ADD CONSTRAINT fk_comments_user
72     FOREIGN KEY (user_id) REFERENCES users(user_id);
73 ALTER TABLE approvals ADD CONSTRAINT fk_approvals_version
74     FOREIGN KEY (version_id) REFERENCES document_versions(
version_id);
75 ALTER TABLE approvals ADD CONSTRAINT fk_approvals_user
76     FOREIGN KEY (user_id) REFERENCES users(user_id);
77 ALTER TABLE grounding_evidences ADD CONSTRAINT
fk_evidences_version
78     FOREIGN KEY (version_id) REFERENCES document_versions(
version_id);
79 ALTER TABLE drift_alerts ADD CONSTRAINT fk_alerts_repository
80     FOREIGN KEY (repository_id) REFERENCES repositories(
repository_id);
81
82 GO
```


Phần IV

Tổng Kết

Chương 6

Tổng Kết

Đề tài đã hoàn thành xuất sắc quy trình thiết kế cơ sở dữ liệu toàn diện cho hệ thống **LivingDocs** trải dài qua ba giai đoạn chuẩn hóa từ Stage 1 đến Stage 3. Ở Mức quan niệm (Stage 1), mô hình ERD và UML Class Diagram đã chuẩn hóa thành công các thực thể nghiệp vụ phức tạp như phân quyền Scoped RBAC, luồng kiểm duyệt 2 cấp và cơ chế lưu minh chứng AI. Chuyển sang Mức logic (Stage 2), hệ thống được ánh xạ chính xác sang Lược đồ quan hệ và chứng minh đạt chuẩn 3NF, giúp loại bỏ triệt để sự dư thừa dữ liệu cũng như các bất thường khi thêm, sửa, xóa. Tại Mức vật lý (Stage 3), các bảng được tối ưu hóa kiểu dữ liệu, hệ thống chỉ mục (Index) và ràng buộc toàn vẹn nhằm nâng cao hiệu năng truy vấn ở mức tối đa. Kết quả thu được tạo nên một nền tảng kiến trúc dữ liệu vững chắc, sẵn sàng đáp ứng khả năng mở rộng và vận hành ổn định cho dự án khi triển khai thực tế. Đây là minh chứng rõ nét cho tính hiệu quả của việc áp dụng quy trình thiết kế CSDL bài bản vào việc xây dựng một hệ thống phần mềm chuyên nghiệp.